

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG



Báo Cáo Bài Tập Lớn

Đề tài: **Hạ Tầng Khóa Công Khai**
(Public Key Infrastructure – PKI)

Học phần: **Nhập môn An toàn thông tin – IT4015**

Mã học phần: IT4015

Mã lớp: 167834

Giảng viên:	PGS. TS. Nguyễn Linh Giang	
Nhóm sinh viên:	Đỗ Trường Giang	20235066
	Hoàng Minh Khôi	20230042
	Phạm Gia Hưng	20230036
	Nguyễn Thanh Tú	20230075

Hà Nội, June 9, 2026

Contents

1	Giới thiệu	1
1.1	Vấn đề đặt ra	1
1.2	Phạm vi và bố cục báo cáo	2
2	Cơ sở Mật mã học và Sự tiến hóa của Hệ thống Khóa	3
2.1	Mật mã đối xứng và hạn chế	3
2.2	Mật mã khóa công khai	6
2.3	Tại sao mật mã bất đối xứng chưa đủ?	20
3	Cấu trúc hạ tầng khóa công khai PKI	23
3.1	Các mô hình phân phối khóa công khai	23
3.1.1	Bài toán phân phối khóa công khai	23
3.1.2	Public Announcement	23
3.1.3	Host-based Trust	24
3.1.4	Web of Trust (PGP)	24
3.1.5	Hierarchical PKI	24
3.1.6	So sánh trade-off	24
3.2	Định nghĩa và mục tiêu PKI	25
3.3	Các thành phần PKI	26
3.4	Chu kỳ sống của chứng chỉ	28
4	Chứng chỉ số và các chuẩn	29
4.1	Chứng chỉ số X.509	29
4.2	Các trường X.509 v3	30
4.3	Distinguished Name	30
4.4	Extension quan trọng	31
4.5	Certificate Signing Request	32
4.6	Phân loại chứng chỉ	32
4.7	Các chuẩn PKCS	33
4.8	RFC và chuẩn quan trọng	33

5	Triển khai và ứng dụng thực tế	34
5.1	HTTPS / TLS	34
5.2	Chữ ký số văn bản	36
5.3	S/MIME	37
5.4	VPN và IPSec	37
5.5	Code Signing	38
5.6	eGovernment Việt Nam	39
6	Các hệ thống PKI mã nguồn mở	40
6.1	Tiêu chí phân loại	40
6.2	OpenSSL	41
6.3	XCA	41
6.4	EJBCA	42
6.5	Let's Encrypt và Certbot	43
6.6	Vault PKI	43
6.7	OAuth/OIDC và PKI	44
6.8	Dogtag và FreeIPA	44
6.9	So sánh hệ sinh thái	45
7	Thách thức và xu hướng	47
7.1	Thách thức hiện tại	47
7.2	Tự động hóa và giám sát	47
7.3	PKI trong hệ thống hiện đại	48
7.4	Post-quantum	48
8	Kết luận	50
8.1	Tổng kết	50
8.2	Ý nghĩa và lời cảm ơn	50
	Tài liệu tham khảo	51

Chapter 1

Giới thiệu

1.1 Vấn đề đặt ra

Mật mã khóa công khai (public-key cryptography) đã mang lại một bước tiến quan trọng trong lĩnh vực an toàn thông tin khi cho phép các thực thể trao đổi dữ liệu bí mật và thực hiện chữ ký số mà không cần phải chia sẻ khóa bí mật từ trước. Nhờ đó, các hệ thống có thể hoạt động trên môi trường mạng mở như Internet mà vẫn đảm bảo được tính bảo mật và toàn vẹn của dữ liệu.

Tuy nhiên, một vấn đề cốt lõi vẫn chưa được giải quyết bởi bản thân mật mã khóa công khai, đó là: làm thế nào để xác định rằng một khóa công khai thực sự thuộc về đúng thực thể mà nó tuyên bố đại diện. Nói cách khác, hệ thống vẫn thiếu một cơ chế để ràng buộc giữa danh tính (identity) và khóa công khai (public key).

Trong môi trường mạng không tin cậy, kẻ tấn công có thể khai thác điểm yếu này để thực hiện tấn công trung gian (Man-in-the-Middle). Ví dụ, khi hai bên A và B trao đổi khóa công khai, kẻ tấn công có thể chặn kết nối và thay thế khóa công khai của B bằng khóa của chính mình. Khi đó, A sẽ mã hóa dữ liệu bằng khóa của kẻ tấn công mà không hề hay biết. Mặc dù các thuật toán mật mã vẫn hoạt động chính xác về mặt kỹ thuật, toàn bộ thông tin trao đổi đã bị lộ và có thể bị chỉnh sửa trước khi đến đích.

Một ví dụ thực tế có thể thấy trong quá trình truy cập website. Nếu không có cơ chế xác thực đáng tin cậy, người dùng có thể bị chuyển hướng đến một website giả mạo có giao diện giống hệt website thật. Website giả này cung cấp một khóa công khai do kẻ tấn công kiểm soát, khiến người dùng vô tình thiết lập kết nối “an toàn” với chính kẻ tấn công.

Do đó, một yêu cầu quan trọng được đặt ra là cần có một cơ chế đáng tin cậy để xác minh rằng một khóa công khai thực sự thuộc về đúng thực thể tương ứng. Cơ chế này phải hoạt động hiệu quả ngay cả trong môi trường mạng mở, nơi không thể tin tưởng các kênh truyền thông. Hạ tầng khóa công khai (Public Key Infrastructure – PKI) được đề xuất nhằm giải quyết chính vấn đề này, bằng cách cung cấp một hệ thống quản lý và xác thực khóa công khai dựa trên các bên thứ ba tin cậy.

1.2 Phạm vi và bố cục báo cáo

Báo cáo này tập trung nghiên cứu về Hạ tầng khóa công khai (Public Key Infrastructure – PKI), với mục tiêu làm rõ vai trò của PKI trong việc đảm bảo an toàn thông tin trong các hệ thống hiện đại. Nội dung báo cáo tiếp cận từ góc độ hệ thống và ứng dụng, không đi sâu vào các chứng minh toán học phức tạp của các thuật toán mật mã, mà tập trung vào việc giải thích nguyên lý hoạt động, kiến trúc và cách thức triển khai trong thực tế.

Cách tiếp cận của báo cáo được tổ chức theo hướng mỗi chương trả lời một câu hỏi cụ thể, giúp người đọc từng bước xây dựng hiểu biết một cách logic và có hệ thống.

Chương 2 trả lời câu hỏi “Tại sao cần PKI?” thông qua việc trình bày các nền tảng của mật mã đối xứng và bất đối xứng, cũng như các hạn chế còn tồn tại của chúng.

Chương 3 tập trung vào câu hỏi “PKI gồm những thành phần nào?”, trong đó mô tả kiến trúc hệ thống và các thực thể như Certification Authority (CA), Registration Authority (RA) và các cơ chế quản lý chứng chỉ.

Chương 4 trả lời câu hỏi “Chứng chỉ số được định nghĩa như thế nào?”, trình bày cấu trúc và vai trò của chứng chỉ số X.509 trong việc ràng buộc giữa danh tính và khóa công khai.

Chương 5 làm rõ “PKI được sử dụng ở đâu trong thực tế?”, với các ví dụ tiêu biểu như HTTPS, chữ ký số và các hệ thống bảo mật khác.

Chương 6 khảo sát các công cụ và nền tảng mã nguồn mở cho phép triển khai PKI trong thực tế, trong khi Chương 7 thảo luận về các thách thức hiện tại cũng như xu hướng phát triển trong tương lai của PKI.

Chapter 2

Cơ sở Mật mã học và Sự tiến hóa của Hệ thống Khóa

Trong kỷ nguyên số, an toàn thông tin được xây dựng trên những nền tảng toán học phức tạp của Mật mã học (Cryptography). Tuy nhiên, lịch sử phát triển của ngành mật mã không phải là một đường thẳng, mà là một quá trình tiến hóa liên tục để giải quyết những nghịch lý phát sinh từ chính các giải pháp trước đó. Sự tiến hóa này bắt nguồn từ mật mã đối xứng với tốc độ vượt trội nhưng vướng mắc ở khâu phân phối khóa; dẫn đến sự ra đời của mật mã bất đối xứng nhằm giải quyết bài toán trao đổi khóa nhưng lại tạo ra lỗ hổng về xác thực danh tính. Cuối cùng, Hạ tầng Khóa công khai (PKI) xuất hiện như một mảnh ghép hoàn thiện, kết nối sức mạnh toán học với cơ chế quản lý niềm tin trong thế giới thực. Chương này sẽ trình bày chi tiết chuỗi logic nền tảng đó.

2.1 Mật mã đối xứng và hạn chế

Nguyên lý hoạt động của Mật mã đối xứng

Mật mã đối xứng (Symmetric-key Cryptography), hay còn gọi là mật mã khóa bí mật, là hình thức mã hóa cổ điển và lâu đời nhất trong lịch sử nhân loại. Nguyên lý cốt lõi của hệ thống này nằm ở tính "đối xứng" của chìa khóa: người gửi và người nhận sử dụng **cùng một khóa duy nhất (K)** cho cả hai quá trình mã hóa (Encryption) và giải mã (Decryption).

Về mặt toán học, quá trình này có thể được biểu diễn thông qua hai hàm cơ bản:

$$C = E(K, P)$$

$$P = D(K, C)$$

Trong đó, P là bản rõ (Plaintext), C là bản mã (Ciphertext), hàm E và D lần lượt là thuật toán mã hóa và giải mã sử dụng khóa bí mật K .

Ưu điểm tuyệt đối và lý do khiến mật mã đối xứng vẫn là trụ cột không thể thay thế trong truyền tải dữ liệu hiện đại chính là **tốc độ xử lý (High Performance)**. Các thuật toán đối xứng tiêu biểu như **DES** (Data Encryption Standard), **3DES** (Triple DES) và đặc biệt là **AES** (Advanced Encryption Standard) được thiết kế dựa trên các phép toán mức bit cực kỳ đơn giản đối với phần cứng máy tính (như phép XOR, dịch chuyển bit Shift, và hoán vị Permutation). Thay vì phải giải quyết các bài toán toán học phức tạp như hệ mật mã bất đối xứng, AES xử lý dữ liệu theo từng khối (Block cipher) với tốc độ có thể lên tới hàng Gigabytes trên giây, đáp ứng hoàn hảo cho việc truyền phát Video, mã hóa ổ cứng hay các luồng dữ liệu thời gian thực.

Bài toán phân phối khóa (The Key Distribution Problem)

Mặc dù sở hữu hiệu năng xuất sắc, mật mã đối xứng lại mang trong mình một tử huyệt mang tính hệ thống khi triển khai trên quy mô lớn: **Bài toán phân phối khóa**.

Nghịch lý của hệ thống đối xứng nằm ở một vòng luẩn quẩn (Catch-22): Để hai bên Alice và Bob có thể giao tiếp an toàn qua mạng Internet, họ cần phải chia sẻ trước với nhau một chiếc khóa K . Tuy nhiên, để gửi chiếc khóa K này cho nhau một cách an toàn mà không bị kẻ thứ ba nghe lén, họ lại cần một kênh liên lạc đã được bảo mật từ trước. Nếu họ đã có sẵn một kênh bảo mật, thì họ đâu cần thiết lập hệ thống mật mã làm gì nữa? Trong quá khứ, giải pháp duy nhất là trao đổi khóa qua một kênh ngoại tuyến (out-of-band channel) như gặp mặt trực tiếp hoặc gửi qua người đưa thư (courier), điều này là hoàn toàn bất khả thi trong kỷ nguyên thương mại điện tử toàn cầu.

Hơn thế nữa, bài toán phân phối khóa tạo ra một rào cản khủng khiếp về mặt mở rộng quy mô (Scalability). Trong một mạng lưới có n thực thể (node), nếu mỗi cặp thực thể đều muốn liên lạc bảo mật với nhau, hệ thống sẽ yêu cầu số lượng khóa phải quản lý tăng theo cấp số nhân. Công thức tính tổng số cặp khóa riêng biệt trong mạng là:

$$\text{Tổng số khóa} = \frac{n(n-1)}{2} = O(n^2)$$

Hãy thử làm một phép tính thực tế:

- Nếu mạng nội bộ chỉ có 10 nhân viên, số khóa cần quản lý là $10 \times 9/2 = 45$ khóa. (Có thể quản lý thủ công).
- Nếu một hệ thống ngân hàng có 1,000 khách hàng, số khóa cần tạo ra là gần 500,000 khóa.
- Nếu triển khai trên phạm vi Internet với 1 triệu người dùng, chúng ta sẽ cần tới xấp xỉ **500 tỷ** chiếc khóa bí mật khác nhau.

Việc lưu trữ, phân phối, cập nhật và thu hồi hàng tỷ chiếc khóa bí mật này tạo ra một cơn ác mộng về mặt quản trị mạng, khiến mô hình đối xứng thuần túy trở nên hoàn toàn

vô dụng đối với môi trường mạng phi tập trung.

Giải pháp KDC (Key Distribution Center) và những giới hạn chí mạng

Để khắc phục bài toán $O(n^2)$ và sự chồng kênh trong việc quản lý khóa, các nhà khoa học máy tính đã đề xuất mô hình **Trung tâm Phân phối Khóa (Key Distribution Center - KDC)**.

Thay vì mỗi người dùng phải ghi nhớ hàng vạn chiếc khóa để nói chuyện với hàng vạn người khác, KDC đóng vai trò là một Bên thứ ba đáng tin cậy (Trusted Third Party). Cơ chế hoạt động của hệ thống được tinh giản như sau:

1. Mỗi người dùng trên mạng chỉ cần chia sẻ **một chiếc khóa chủ (Master Key) duy nhất** với KDC. Lúc này số lượng khóa hệ thống phải quản lý giảm từ $O(n^2)$ xuống chỉ còn $O(n)$.
2. Khi Alice muốn liên lạc với Bob, Alice không liên hệ trực tiếp với Bob mà gửi yêu cầu lên KDC.
3. KDC sẽ sinh ra một chiếc **Khóa phiên ngẫu nhiên (Session Key)**. KDC mã hóa chiếc khóa phiên này bằng khóa chủ của Alice rồi gửi cho Alice; đồng thời mã hóa chiếc khóa phiên đó bằng khóa chủ của Bob rồi gửi cho Bob.
4. Alice và Bob giải mã để lấy Khóa phiên, và sử dụng nó để nhắn tin trực tiếp với nhau. Khi phiên liên lạc kết thúc, chiếc khóa phiên này sẽ bị hủy bỏ.

Giao thức nổi tiếng nhất triển khai mô hình KDC này chính là **Kerberos** [1], được sử dụng rộng rãi trong môi trường mạng nội bộ của doanh nghiệp (như Windows Active Directory). Nó giải quyết xuất sắc bài toán mở rộng quy mô.

Tuy nhiên, KDC không phải là viên đạn bạc. Nó mang trong mình những **giới hạn chí mạng** khiến nó không thể trở thành chuẩn mực cho môi trường Internet mở:

- **Điểm lỗi tập trung (Single Point of Failure - SPOF):** KDC là trái tim của toàn bộ mạng lưới. Nếu máy chủ KDC bị sập (do lỗi phần cứng, mất điện, hoặc bị tấn công DDoS), không một ai trong hệ thống có thể thiết lập được phiên liên lạc mới. Toàn bộ mạng lưới sẽ bị tê liệt hoàn toàn.
- **Nút thắt cổ chai hiệu năng (Performance Bottleneck):** Mọi yêu cầu liên lạc mới của hàng triệu người dùng đều phải dội ngược về KDC để xin cấp Session Key. Điều này đòi hỏi KDC phải có năng lực xử lý khổng lồ, rất dễ gây ra tắc nghẽn mạng (Network congestion).
- **Thảm họa bảo mật (God-mode vulnerability):** KDC là "chiếc rương" chứa mọi khóa chủ của tất cả người dùng. Nếu một nhóm hacker (hoặc nội gián) xâm

nhập thành công vào máy chủ KDC, chúng có khả năng nghe lén, giải mã, hoặc giả mạo bất kỳ thông điệp nào của bất kỳ ai trong hệ thống. Cả mạng lưới sẽ sụp đổ về mặt an toàn thông tin.

- **Vấn đề về ranh giới niềm tin (Trust Boundary):** KDC yêu cầu mọi thành viên phải tin tưởng tuyệt đối vào cơ quan quản lý KDC. Điều này hoạt động tốt trong một công ty (nhân viên tin tưởng phòng IT). Nhưng trên môi trường Internet mở, nơi Alice ở Việt Nam muốn mua hàng của Bob ở Mỹ, không tồn tại một tổ chức KDC chung nào mà cả hai quốc gia và cả hai cá nhân đều sẵn sàng giao phó toàn bộ khóa bí mật của mình.

Sự bế tắc của KDC trong việc thiết lập niềm tin trên diện rộng đã đặt ra một yêu cầu bức thiết cho ngành Mật mã học: Cần phải có một hệ thống cho phép hai người hoàn toàn xa lạ giao tiếp bảo mật với nhau mà không cần phải chia sẻ trước bất kỳ bí mật nào, cũng không cần phải trao quyền sinh sát cho một trung tâm KDC tập trung. Và đó chính là lúc Mật mã khóa bất đối xứng (Asymmetric Cryptography) bước ra ánh sáng.

2.2 Mật mã khóa công khai

Cặp khóa bất đối xứng và Trapdoor function

Khác với hệ mật mã đối xứng chỉ sử dụng một khóa duy nhất, mật mã khóa công khai (Public-key Cryptography) hoạt động dựa trên một cặp khóa gắn kết chặt chẽ về mặt toán học: **Khóa công khai (Public Key - PU)** và **Khóa bí mật (Private Key - PR)**. Tùy thuộc vào cách phối hợp và thứ tự sử dụng hai chiếc khóa này, hệ thống có thể giải quyết ba bài toán cốt lõi của an toàn thông tin:

1. **Đảm bảo tính bảo mật (Confidentiality):** Người gửi mã hóa thông điệp bằng *PU* của người nhận. Lúc này, bản mã chỉ có thể được giải mã bằng *PR* duy nhất mà người nhận đang cất giữ. Kịch bản này bảo vệ dữ liệu khỏi những kẻ nghe lén trên đường truyền.
2. **Xác thực và Chữ ký số (Authentication/Digital Signature):** Người gửi "mã hóa" (hoặc ký) thông điệp bằng chính *PR* của mình. Bất kỳ ai trên mạng cũng có thể dùng *PU* của người gửi để giải mã. Nếu quá trình giải mã thành công, điều này minh chứng tuyệt đối rằng thông điệp thực sự xuất phát từ người gửi đó và nội dung không hề bị chỉnh sửa (chống chối cãi).
3. **Bảo mật và Xác thực song hành:** Người gửi kết hợp cả hai thao tác. Đầu tiên, ký thông điệp bằng *PR* của nguồn gửi để xác thực, sau đó tiếp tục bọc thêm một lớp mã hóa bằng *PU* của đích đến để bảo mật. Nhờ vậy, thông điệp vừa được giữ kín, vừa đảm bảo tính chính danh.

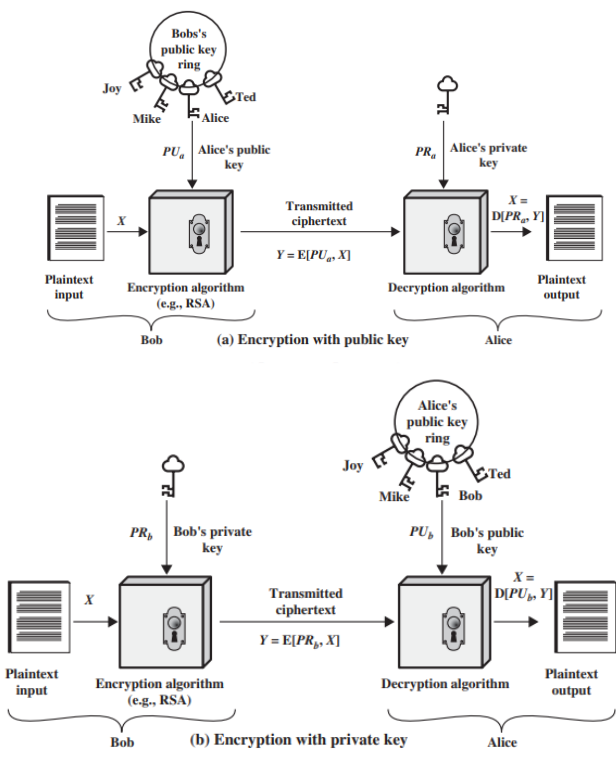


Figure 9.1 Public-Key Cryptography

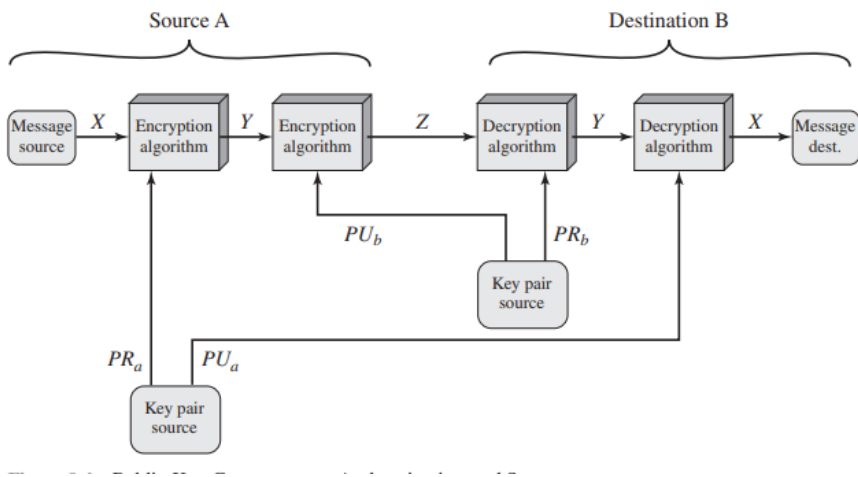


Figure 2.1: Ba kịch bản ứng dụng cốt lõi của hệ mật mã khóa bất đối xứng

Kịch bản 1: Đảm bảo tính bảo mật (Encryption with Public Key) Mục tiêu của kịch bản này là đảm bảo chỉ duy nhất người nhận hợp pháp mới có thể đọc được nội dung thông điệp (Confidentiality).

- **Phía người gửi (Bob):** Bob lấy thông điệp gốc X và tra cứu Khóa công khai của người nhận (Alice) trên một vòng khóa (public key ring). Sau đó, Bob thực hiện mã hóa:

$$Y = E[PU_a, X]$$

Bản mã Y lúc này được truyền đi trên kênh không an toàn.

- **Phía người nhận (Alice):** Khi nhận được Y , Alice sử dụng Khóa bí mật duy nhất của mình (PR_a) để giải mã:

$$X = D[PR_a, Y]$$

Vì chỉ có Alice mới nắm giữ PR_a , mọi kẻ nghe lén dù bắt được Y và biết PU_a cũng không thể đảo ngược quá trình để tìm ra X .

Kịch bản 2: Xác thực nguồn gốc và Chữ ký số (Encryption with Private Key) Mục tiêu ở đây không phải là giấu kín dữ liệu, mà là chứng minh tuyệt đối nguồn gốc của thông điệp (Authentication).

- **Phía người gửi (Bob):** Bob dùng chính Khóa bí mật của mình (PR_b) để "mã hóa" (bản chất là ký số) lên thông điệp X :

$$Y = E[PR_b, X]$$

- **Phía người nhận (Alice):** Alice nhận Y và tra cứu Khóa công khai của Bob (PU_b) để khôi phục:

$$X = D[PU_b, Y]$$

Bất kỳ ai trên mạng cũng có thể dùng PU_b để đọc được thông điệp này. Tuy nhiên, nếu hàm giải mã D chạy thành công và trả về dữ liệu có nghĩa, điều đó minh chứng 100% rằng bản mã Y *bắt buộc* phải được tạo ra từ PR_b của Bob. Bob không thể chối cãi việc mình đã gửi thông điệp này.

Kịch bản 3: Bảo mật và Xác thực song hành (Secrecy and Authentication) Để đáp ứng cả hai yêu cầu khắt khe nhất trong liên lạc, hệ thống sẽ bọc dữ liệu qua hai lớp vỏ mã hóa, kết hợp cả hai kịch bản trên.

- **Phía người gửi (Source A):** Đầu tiên, A tạo chữ ký để xác thực bằng cách mã hóa với Khóa bí mật của mình: $Y = E[PR_a, X]$. Sau đó, A bọc lớp bảo mật bên

ngoài bằng cách mã hóa tiếp Y với Khóa công khai của đích đến B:

$$Z = E[PU_b, Y]$$

- **Phía người nhận (Destination B):** Quá trình bóc tách diễn ra ngược lại. B dùng Khóa bí mật của mình để phá lớp vỏ bảo mật: $Y = D[PR_b, Z]$. Cuối cùng, B dùng Khóa công khai của A để xác thực và lấy ra thông điệp gốc:

$$X = D[PU_a, Y]$$

Nền tảng toán học: Hàm một chiều có cửa sập (Trapdoor One-way Function)

Để các kịch bản trên có thể hoạt động an toàn trong thực tế, hệ thống đòi hỏi một cơ chế toán học cực kỳ khắt khe. Việc công khai PU và thuật toán mã hóa cho cả thế giới biết đặt ra thách thức: Làm sao để ngăn chặn hacker tính ngược lại để tìm ra PR hoặc bản rõ ban đầu? Câu trả lời nằm ở khái niệm **Hàm một chiều có cửa sập**.

Khái niệm này bao gồm hai tính chất then chốt:

- **Hàm một chiều (One-way Function):** Là một hàm số $Y = f(X)$ thỏa mãn điều kiện: nếu biết X , việc tính xuôi ra Y là rất dễ dàng (thời gian đa thức). Tuy nhiên, nếu chỉ biết Y , việc tính ngược $X = f^{-1}(Y)$ là *bất khả thi về mặt tính toán* (*computationally infeasible*) đối với máy tính hiện hành.
- **Cửa sập (Trapdoor):** Hệ mật mã không thể chỉ dùng hàm một chiều đơn thuần (vì chính người nhận cũng sẽ không giải mã được). Do đó, hàm này được thiết kế thêm một "cửa sập". Nếu sở hữu thông tin bổ sung k (chính là Khóa bí mật PR), người nhận có thể kích hoạt cửa sập và dễ dàng tính toán ngược $X = f_k^{-1}(Y)$.

Tóm lại, sự sinh tồn của mật mã bất đối xứng phụ thuộc vào việc xây dựng thành công hàm tính toán một chiều vững chắc, nơi Khóa công khai đóng vai trò là hàm f để chốt khóa, còn Khóa bí mật chính là chìa khóa cửa sập k để mở luồng tính ngược.

Thuật toán RSA

Thuật toán RSA, được đặt theo tên của ba nhà phát minh Rivest, Shamir và Adleman, là một trong những hệ mật mã khóa công khai phổ biến và nền tảng nhất. Tính an toàn của RSA dựa trên độ khó của **bài toán phân tích thừa số nguyên tố lớn (Integer Factorization Problem)** [2]. Về mặt toán học, việc nhân hai số nguyên tố khổng lồ để tạo ra một hợp số thì diễn ra rất nhanh, nhưng việc làm ngược lại — phân tích hợp số đó ra hai thừa số ban đầu — là một bài toán bất khả thi (infeasible) đối với khả năng điện toán của máy tính hiện tại.

Quy trình hoạt động của RSA Hệ thống RSA được vận hành qua 3 giai đoạn cốt lõi:

1. **Tạo khóa (Key Generation):** (Thực hiện bởi người nhận - Alice)

- Chọn hai số nguyên tố lớn ngẫu nhiên p và q ($p \neq q$).
- Tính modulus $n = p \times q$.
- Tính giá trị hàm Euler totient: $\phi(n) = (p - 1)(q - 1)$.
- Chọn một số nguyên e sao cho $1 < e < \phi(n)$ và $\gcd(\phi(n), e) = 1$ (tức là e và $\phi(n)$ nguyên tố cùng nhau).
- Tính khóa giải mã d là nghịch đảo của e theo modulo $\phi(n)$, tức là:

$$d \equiv e^{-1} \pmod{\phi(n)}$$

- **Khóa công khai (Public Key):** $PU = \{e, n\}$.
- **Khóa bí mật (Private Key):** $PR = \{d, n\}$.
(Lưu ý: Các giá trị p, q và $\phi(n)$ phải được tiêu hủy hoặc cất giữ tuyệt mật sau khi tạo khóa xong).

2. **Mã hóa (Encryption):** (Thực hiện bởi người gửi - Bob)

- Bob muốn gửi thông điệp M cho Alice (với điều kiện $M < n$).
- Bob sử dụng Khóa công khai PU của Alice để tính bản mã C :

$$C = M^e \bmod n$$

3. **Giải mã (Decryption):** (Thực hiện bởi người nhận - Alice)

- Alice nhận bản mã C và sử dụng Khóa bí mật PR của mình để khôi phục nguyên trạng thông điệp M :

$$M = C^d \bmod n$$

Tính bảo mật của RSA Độ an toàn của hệ mật mã RSA phải đối mặt với 4 hướng tấn công chủ đạo từ giới phân tích mật mã (cryptanalysis):

1. **Tấn công vét cạn (Brute force):** Thử tất cả các khóa bí mật (Private Key) có thể. Hướng này bị vô hiệu hóa bằng cách sử dụng không gian khóa khổng lồ.
2. **Tấn công toán học (Mathematical attacks):** Tập trung vào việc giải bài toán phân tích thừa số nguyên tố lớn để dịch ngược các thông số cốt lõi.

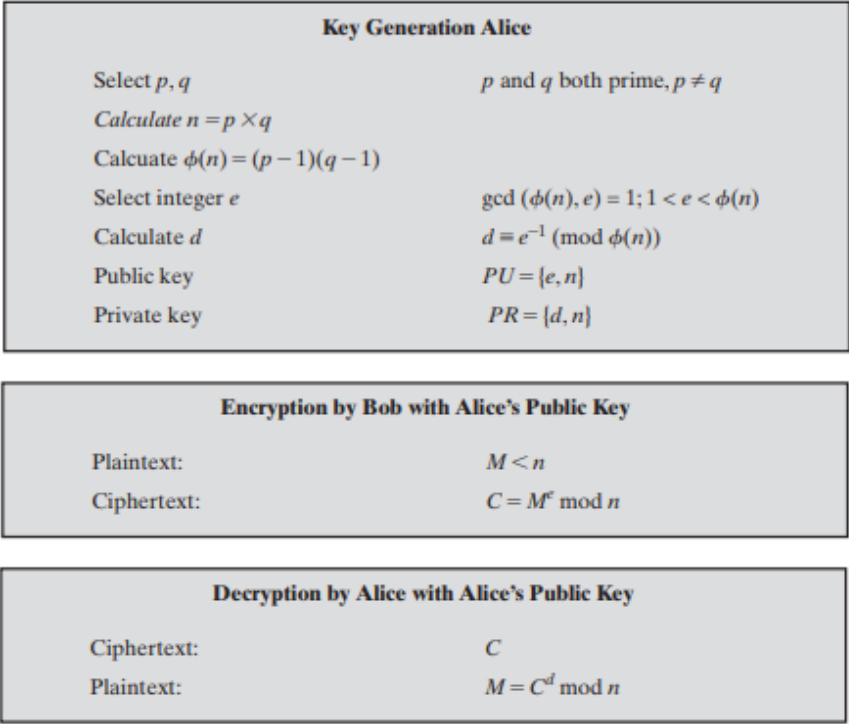


Figure 9.5 The RSA Algorithm

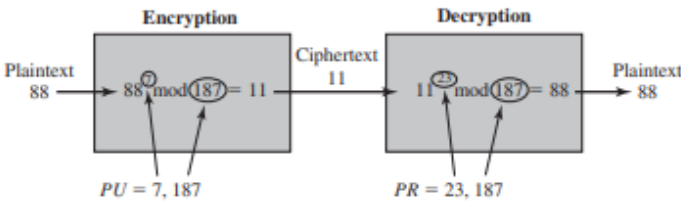


Figure 9.6 Example of RSA Algorithm

Figure 2.2: Sơ đồ tạo khóa, mã hóa và giải mã của thuật toán RSA (Nguồn: Figure 9.5 & 9.6)

3. **Tấn công thời gian (Timing attacks):** Một dạng tấn công kênh kề (side-channel), đo lường thời gian chạy của thuật toán giải mã để suy luận ra từng bit của khóa bí mật.
4. **Tấn công bản mã lựa chọn (Chosen Ciphertext attacks - CCA):** Khai thác các tính chất toán học cơ bản của thuật toán RSA để tạo ra các thông điệp có chủ đích.

1. Bài toán phân tích thừa số (The Factoring Problem)

Phòng tuyến toán học đầu tiên và quan trọng nhất của RSA là khó khăn trong việc phân tích một hợp số lớn n thành hai thừa số nguyên tố p và q . Nếu kẻ tấn công có thể phân tích thành công n , chúng sẽ dễ dàng tính được $\phi(n) = (p - 1)(q - 1)$ và từ đó trích xuất được khóa giải mã d . Mặc dù các thuật toán phân tích (như General Number Field Sieve - GNFS) ngày càng tiến bộ, nhưng nỗ lực tính toán vẫn tăng theo cấp số mũ so với độ dài của khóa. Do đó, việc nâng kích thước khóa RSA lên tiêu chuẩn 2048-bit hoặc 4096-bit được coi là an toàn tuyệt đối trước các đòn tấn công toán học truyền thống.

2. Tấn công thời gian (Timing Attack) và Cách khắc phục

Timing Attack là một kịch bản tấn công thực tế và cực kỳ nguy hiểm do Paul Kocher đề xuất. Kẻ tấn công không cần giải toán mà sẽ đo lường thời gian phản hồi (chậm/nhanh) của bộ xử lý khi thực thi thuật toán lũy thừa modulo (Square and Multiply). Sự chênh lệch thời gian xử lý giữa bit '0' và bit '1' của khóa d sẽ tố cáo toàn bộ giá trị của khóa.

Để chống lại đòn tấn công này, có 3 biện pháp kỹ thuật được áp dụng:

- **Cố định thời gian tính toán (Constant exponentiation time):** Buộc hệ thống luôn phải tiêu tốn một khoảng thời gian cố định cho mọi phép tính, bất kể bit đó là '0' hay '1'. Tuy nhiên, điều này làm giảm hiệu năng hệ thống.
- **Thêm độ trễ ngẫu nhiên (Random delay):** Chèn thêm các khoảng thời gian chờ ngẫu nhiên vào thuật toán để làm nhiễu thông số đo lường của kẻ tấn công.
- **Tung hỏa mù (Blinding):** Đây là biện pháp tối ưu nhất. Trước khi giải mã, bản mã C được nhân với một giá trị ngẫu nhiên r : $C' = C \cdot r^e \bmod n$. Bộ xử lý sẽ giải mã trên một cục dữ liệu rác C' , khiến mọi phép đo thời gian của hacker trở nên vô nghĩa, sau đó hệ thống sẽ khử r ở bước cuối cùng để trả về bản rõ M .

3. Tấn công CCA và chuẩn OAEP (Optimal Asymmetric Encryption Padding)

Thuật toán RSA nguyên thủy (Basic RSA) có một lỗ hổng nghiêm trọng dựa trên đặc tính nhân (multiplicative property): Mã hóa của M_1 nhân với mã hóa của M_2 sẽ bằng mã hóa của $(M_1 \times M_2)$. Kẻ tấn công có thể lợi dụng điều này bằng cách chèn thêm các hệ số

nhập vào bản mã C , biến đổi nó thành một bản mã có chủ đích (Chosen Ciphertext) và lừa máy chủ giải mã để lấy thông tin.

Để vô hiệu hóa hoàn toàn các phương pháp tấn công này, hệ thống RSA thực tế không bao giờ mã hóa trực tiếp thông điệp M , mà phải đi qua một cơ chế "đệm" cực kỳ phức tạp có tên là **OAEP (Optimal Asymmetric Encryption Padding)**.

Về bản chất, OAEP sẽ trộn lẫn thông điệp M ban đầu với các chuỗi padding, các giá trị băm $H(P)$ và một hạt giống ngẫu nhiên (Seed). Thông qua mạng lưới kết hợp các hàm sinh mặt nạ (Mask Generating Function - MGF) và phép toán XOR chéo, OAEP sẽ nhào nặn thông điệp thành một khối dữ liệu (Encoded Message - EM) hoàn toàn hỗn độn và không còn bất kỳ mối liên hệ toán học tuyến tính nào. Nếu kẻ tấn công cố tình nhân thêm hệ số vào bản mã, khi giải mã, cấu trúc OAEP sẽ bị phá vỡ, máy chủ sẽ phát hiện lỗi và từ chối trả về kết quả.

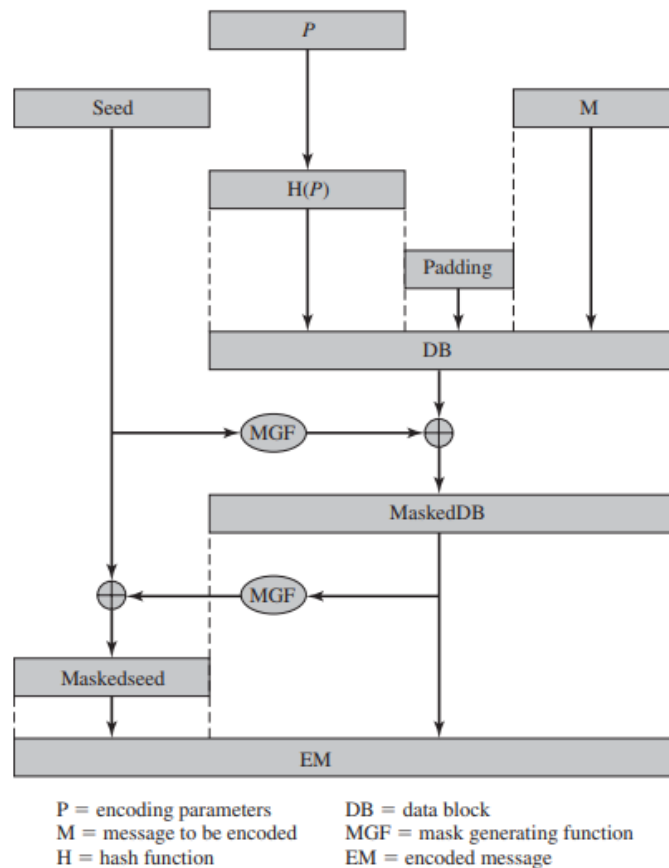


Figure 9.10 Encryption Using Optimal Asymmetric Encryption Padding (OAEP)

Figure 2.3: Sơ đồ khối quá trình nhào nặn dữ liệu của chuẩn OAEP (Nguồn: Figure 9.10)

Trao đổi khóa Diffie-Hellman

Diffie-Hellman (DH) là một trong những hệ thống mật mã khóa công khai (Public-key Cryptography) đầu tiên được công bố. Khác với tính đa dụng của RSA, thuật toán

DH được thiết kế với một mục đích duy nhất và chuyên biệt: **Trao đổi khóa (Key Exchange)**. Sứ mệnh cốt lõi của DH là cho phép hai thực thể hoàn toàn xa lạ có thể đàm phán và thiết lập một Khóa bí mật chung (Shared Secret / Session Key) một cách an toàn thông qua một kênh truyền tải không bảo mật, nơi mọi thông điệp đều có thể bị kẻ tấn công nghe lén. Khóa chung này sau đó sẽ được sử dụng làm đầu vào cho các thuật toán mã hóa đối xứng (như AES) nhằm truyền tải dữ liệu ở tốc độ cao.

Về mặt toán học, độ an toàn của thuật toán Diffie-Hellman phụ thuộc hoàn toàn vào độ khó của **Bài toán Logarit rời rạc (Discrete Logarithm Problem)**. Thuật toán sử dụng số nguyên tố p rất lớn và một căn nguyên thủy (primitive root) a của nó. Trong không gian số học modulo p , việc tính toán xuôi $a^i \bmod p$ là một thao tác cực kỳ nhanh chóng. Tuy nhiên, thao tác tính toán ngược — tức là tìm lại số mũ bí mật i khi chỉ biết kết quả b , cơ số a và số modulo p ($i = \log_a b \pmod{p}$) — là một bài toán bất khả thi về mặt thời gian (infeasible) đối với các hệ thống máy tính hiện tại.

Lỗ hổng bảo mật: Mặc dù giao thức DH nguyên thủy vô hiệu hóa hoàn toàn khả năng giải mã của kẻ nghe lén thụ động (Passive Eavesdropping), nó lại tồn tại một điểm yếu chí mạng: Không có cơ chế xác thực danh tính. Do đó, thuật toán này cực kỳ dễ bị khai thác bởi đòn tấn công **Man-in-the-Middle (MitM)**. Để khắc phục trong các ứng dụng thực tế (như giao thức TLS/SSL), Diffie-Hellman bắt buộc phải được sử dụng kết hợp với Chữ ký số (Digital Signatures) và Chứng chỉ số (Digital Certificates) để đảm bảo tính xác thực của các bên tham gia.

Các bước thực hiện thuật toán Diffie-Hellman Giả sử hai người dùng A (Alice) và B (Bob) muốn thiết lập một khóa bí mật chung qua một kênh truyền không an toàn. Thuật toán được thực hiện qua 3 bước cốt lõi:

1. Khởi tạo thông số công khai (Global Public Elements):

Trước khi trao đổi, Alice và Bob phải thống nhất hai thông số nền tảng (các giá trị này có thể truyền công khai):

- q : Một số nguyên tố rất lớn.
- α : Một căn nguyên thủy (primitive root) của q , với điều kiện $\alpha < q$.

2. Tạo khóa cá nhân và khóa công khai (Key Generation):

• Phía Alice:

- Chọn một số nguyên ngẫu nhiên bí mật $X_A < q$ (đóng vai trò là Private Key).
- Tính giá trị trộn công khai $Y_A = \alpha^{X_A} \bmod q$ và gửi Y_A cho Bob.

• Phía Bob:

- Tương tự, chọn một số nguyên ngẫu nhiên bí mật $X_B < q$.

– Tính giá trị trộn công khai $Y_B = \alpha^{X_B} \bmod q$ và gửi Y_B cho Alice.

3. Tính toán khóa chung bí mật (Calculation of Secret Key):

Sau khi nhận được giá trị Y từ đối phương, hai bên tiến hành tính toán khóa phiên (Session Key) K độc lập tại hệ thống của mình:

- Alice tính: $K = (Y_B)^{X_A} \bmod q$
- Bob tính: $K = (Y_A)^{X_B} \bmod q$

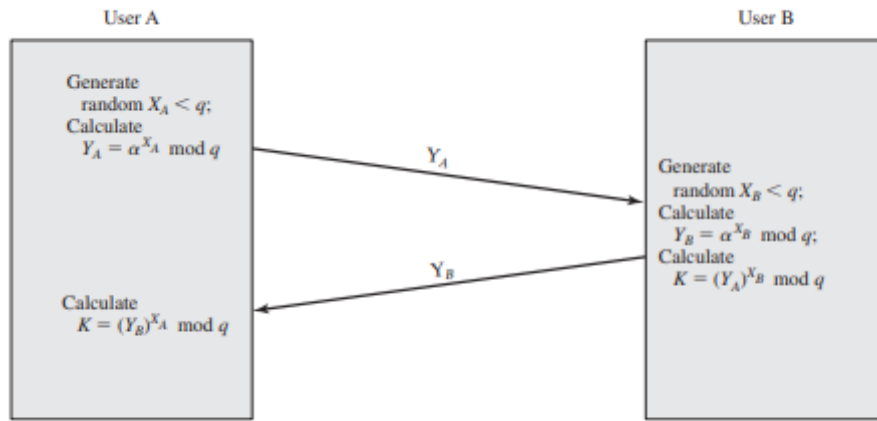


Figure 10.2 Diffie-Hellman Key Exchange

Figure 2.4: Sơ đồ luồng trao đổi khóa Diffie-Hellman (Nguồn: Figure 10.2)

Chứng minh tính đồng nhất của khóa K Dựa vào các tính chất cơ bản của số học modulo, hai biểu thức tính toán độc lập trên thực chất đều hội tụ về một kết quả duy nhất:

$$K = (Y_B)^{X_A} \bmod q = (\alpha^{X_B} \bmod q)^{X_A} \bmod q = \alpha^{X_B \cdot X_A} \bmod q$$

$$K = (Y_A)^{X_B} \bmod q = (\alpha^{X_A} \bmod q)^{X_B} \bmod q = \alpha^{X_A \cdot X_B} \bmod q$$

Do phép nhân số mũ có tính chất giao hoán ($X_A \cdot X_B = X_B \cdot X_A$), cả Alice và Bob đều tổng hợp ra được một chiếc khóa K giống hệt nhau mà không cần phải gửi trực tiếp K qua mạng.

Phân tích độ an toàn trước kẻ tấn công Giả sử có một kẻ nghe lén (Eavesdropper) kiểm soát toàn bộ kênh truyền, kẻ này sẽ thu thập được đầy đủ 4 thông số công khai: q, α, Y_A và Y_B . Tuy nhiên, để tìm ra được khóa chung K , kẻ tấn công bắt buộc phải trích xuất được số mũ bí mật X_A hoặc X_B . Quá trình này đòi hỏi kẻ tấn công phải giải phương trình:

$$X_B = \text{dlog}_{\alpha, q}(Y_B)$$

Trong đó $dlog$ là phép tính logarit rời rạc. Nếu tham số q được chọn đủ lớn (theo tiêu chuẩn hiện tại là từ 2048-bit trở lên), sức mạnh điện toán hiện tại của nhân loại là không thể giải quyết bài toán tính ngược này trong thời gian khả thi (computationally infeasible).

Điểm yếu chí mạng: Tấn công Man-in-the-Middle (MitM) Mặc dù giao thức Diffie-Hellman an toàn tuyệt đối trước những kẻ nghe lén thụ động, nó lại hoàn toàn bất lực trước đòn tấn công chủ động can thiệp vào đường truyền, điển hình là **Tấn công Kẻ đứng giữa (Man-in-the-Middle - MitM)**. Lỗ hổng cốt lõi xuất phát từ việc thuật toán không có cơ chế xác thực danh tính (Authentication) của các bên tham gia.

Kịch bản tấn công MitM:

Giả sử kẻ tấn công (Darth) kiểm soát được đường truyền giữa Alice và Bob. Quá trình đánh chặn diễn ra như sau:

1. **Khởi tạo khóa giả mạo:** Darth tự tạo ra hai bộ khóa bí mật ngẫu nhiên X_{D1} và X_{D2} , từ đó tính ra hai giá trị công khai giả mạo tương ứng là Y_{D1} và Y_{D2} .
2. **Đánh chặn và tráo đổi:**
 - Khi Alice gửi Y_A cho Bob, Darth đánh chặn Y_A , tự động gửi Y_{D1} cho Bob. Lúc này Darth tính được khóa chung thứ nhất: $K2 = (Y_A)^{X_{D2}} \bmod q$.
 - Khi Bob gửi Y_B cho Alice, Darth tiếp tục đánh chặn Y_B , gửi Y_{D2} cho Alice. Darth tính được khóa chung thứ hai: $K1 = (Y_B)^{X_{D1}} \bmod q$.
3. **Ảo giác hệ thống:**
 - Bob nhận Y_{D1} (tưởng của Alice) và tính ra khóa $K1$. Thực chất Bob đang chia sẻ khóa $K1$ với Darth.
 - Alice nhận Y_{D2} (tưởng của Bob) và tính ra khóa $K2$. Thực chất Alice đang chia sẻ khóa $K2$ với Darth.
4. **Thao túng dữ liệu:** Mọi luồng liên lạc trong tương lai đều bị Darth kiểm soát. Nếu Alice gửi thông điệp M mã hóa bằng $K2$, Darth có thể dùng $K2$ để giải mã, đọc trộm hoặc chỉnh sửa thành M' , sau đó mã hóa lại bằng $K1$ và gửi cho Bob. Cả Alice và Bob đều không thể phát hiện ra sự can thiệp này.

Giải pháp khắc phục:

Để giao thức Diffie-Hellman có thể hoạt động an toàn trên thực tế (ví dụ: trong giao thức SSL/TLS), nó bắt buộc phải được kết hợp với các cơ chế định danh như **Chữ ký số (Digital Signatures)** và **Chứng chỉ số khóa công khai (Public-key Certificates)** để đảm bảo các gói tin Y_A và Y_B không bị đánh tráo giữa chừng.

Sơ đồ chữ ký số (Digital Signatures)

Trong các hệ thống xác thực sử dụng khóa đối xứng (Symmetric Authentication), việc trao đổi thông điệp giữa hai bên Alice và Bob luôn tiềm ẩn những nguy cơ tranh chấp pháp lý mà các kỹ thuật mã hóa thông thường không thể giải quyết triệt để. Để hiểu rõ tại sao chúng ta cần đến Chữ ký số, hãy xem xét hai kịch bản rủi ro sau đây: **1. Kịch bản giả mạo nội dung (Forgery by Receiver):**

Giả sử Alice và Bob chia sẻ một khóa bí mật chung để xác thực các giao dịch chuyển tiền. Bob (người nhận) có thể tự ý chỉnh sửa con số trong thông điệp từ 100 USD thành 1000 USD, sau đó dùng khóa chung đó để tạo lại mã xác thực mới. Khi có tranh chấp, Bob có thể khẳng định trước tòa rằng Alice đã gửi 1000 USD, và vì mã xác thực là hợp lệ, Alice không có bằng chứng nào để minh oan.

2. Kịch bản chối bỏ trách nhiệm (Denial of Sending):

Alice gửi một yêu cầu đặt lệnh mua cổ phiếu cho Bob. Tuy nhiên, sau đó thị trường lao dốc khiến Alice thua lỗ. Lúc này, Alice hoàn toàn có thể tuyên bố rằng mình chưa từng gửi thông điệp đó. Vì Bob cũng nắm giữ khóa bí mật chung, Alice có thể đổ lỗi rằng Bob đã tự tạo ra thông điệp để hãm hại mình.

13.1 / DIGITAL SIGNATURES 397

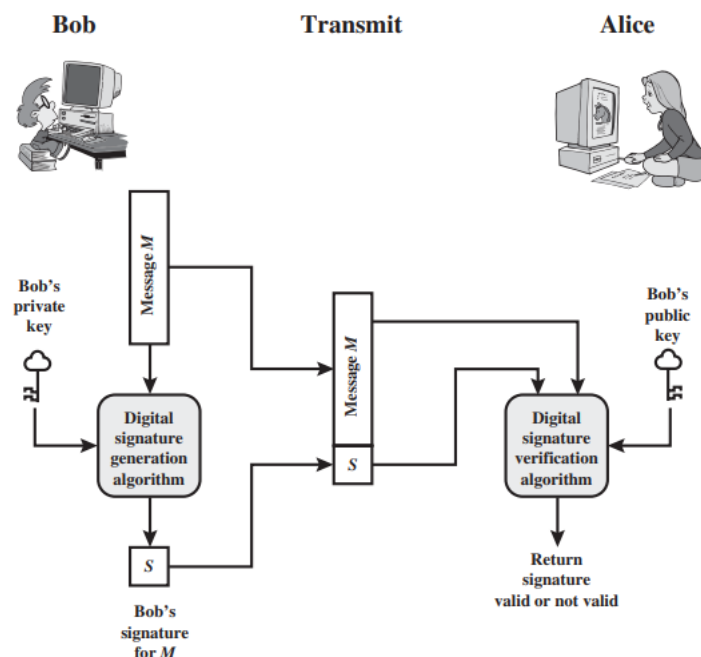


Figure 13.1 Generic Model of Digital Signature Process

For example, suppose that John sends an authenticated message to Mary, using one of the schemes of Figure 12.1. Consider the following disputes that could arise.

Figure 2.5: Mô hình tổng quát của quá trình ký số và các kịch bản tranh chấp phát sinh (Nguồn: Figure 13.1)

Yêu cầu đối với một Chữ ký số tiêu chuẩn Từ những rủi ro trên, chúng ta rút ra kết luận rằng trong các tình huống mà sự tin tưởng giữa người gửi và người nhận không phải là tuyệt đối, chúng ta cần một cơ chế mạnh mẽ hơn xác thực thông thường. Một sơ đồ chữ ký số (Digital Signature) đúng nghĩa phải thỏa mãn các đặc tính sau:

- **Xác thực tác giả và thời điểm:** Phải có khả năng kiểm chứng chính xác ai là người ký và chữ ký đó được tạo ra vào thời gian nào.
- **Xác thực nội dung:** Tại thời điểm ký, nội dung thông điệp phải được "khóa" lại. Mọi sự thay đổi sau đó đều phải làm cho chữ ký trở nên vô hiệu.
- **Khả năng kiểm chứng bởi bên thứ ba:** Đây là đặc tính quan trọng nhất để giải quyết tranh chấp. Một bên thứ ba độc lập (ví dụ: Trọng tài hoặc Tòa án) phải có khả năng kiểm tra tính hợp lệ của chữ ký mà không cần nắm giữ khóa bí mật của người ký.

Như vậy, về mặt chức năng, chữ ký số bao hàm cả chức năng xác thực thông thường nhưng được nâng cấp thêm khả năng **Chống chối cãi (Non-repudiation)**, biến nó thành một công cụ pháp lý quan trọng trong giao dịch điện tử.

Các hình thức tấn công và mức độ giả mạo chữ ký số Dựa trên nghiên cứu của Goldwasser, Micali và Rivest **GOLD88**, các hình thức tấn công vào hệ thống chữ ký số được phân loại theo mức độ nghiêm trọng tăng dần, dựa trên lượng thông tin mà kẻ tấn công (C) có thể thu thập được từ người dùng hợp pháp (A):

- **Tấn công chỉ dựa trên khóa (Key-only attack):** C chỉ biết được khóa công khai của A . Đây là mức độ cơ bản nhất.
- **Tấn công thông điệp đã biết (Known message attack):** C có danh sách một số thông điệp và chữ ký tương ứng của chúng đã được tạo ra trước đó bởi A .
- **Tấn công chọn thông điệp chung (Generic chosen message attack):** C chọn một danh sách các thông điệp trước khi thực hiện tấn công (độc lập với khóa công khai của A) và yêu cầu A ký vào các thông điệp này.
- **Tấn công chọn thông điệp định hướng (Directed chosen message attack):** Tương tự như tấn công chung, nhưng danh sách thông điệp được C chọn sau khi đã biết khóa công khai của A .
- **Tấn công chọn thông điệp thích ứng (Adaptive chosen message attack):** Đây là hình thức nguy hiểm nhất. C sử dụng A như một "oracle" để yêu cầu ký các thông điệp mới dựa trên kết quả của những cặp thông điệp-chữ ký đã thu thập được trước đó.

Song song với các kiểu tấn công, sự thành công của kẻ tấn công trong việc phá vỡ hệ thống (Breaking a signature scheme) cũng được chia thành các cấp độ từ "sụp đổ hoàn toàn" đến "gây nhiễu":

- **Phá vỡ hoàn toàn (Total break):** Kẻ tấn công tìm ra được khóa bí mật cá nhân của A . Đây là thất bại nghiêm trọng nhất, làm sụp đổ toàn bộ hệ thống.
- **Giả mạo phổ quát (Universal forgery):** C tìm ra một thuật toán ký hiệu quả có khả năng tạo ra chữ ký hợp lệ cho bất kỳ thông điệp ngẫu nhiên nào, tương đương với việc sở hữu khóa bí mật.
- **Giả mạo có chọn lọc (Selective forgery):** C tạo ra được chữ ký hợp lệ cho một thông điệp cụ thể mà C đã nhắm mục tiêu từ trước.
- **Giả mạo tồn tại (Existential forgery):** C tạo ra được ít nhất một cặp thông điệp-chữ ký hợp lệ. Tuy nhiên, C không kiểm soát được nội dung của thông điệp này (thông điệp có thể là một chuỗi vô nghĩa), nên đôi khi nó chỉ dừng lại ở mức độ gây nhiễu cho hệ thống.

Yêu cầu kỹ thuật đối với một hệ thống Chữ ký số an toàn Dựa trên việc phân tích các hình thức tấn công và nguy cơ giả mạo (Attacks and Forgeries), chúng ta có thể thiết lập một bộ quy tắc khắt khe để đảm bảo tính sinh tồn của một sơ đồ chữ ký số trong thực tế. Các yêu cầu này không chỉ nhắm vào tính bảo mật mà còn chú trọng đến hiệu năng vận hành:

- **Sự phụ thuộc vào nội dung (Message Dependency):** Chữ ký số phải là một chuỗi bit được tạo ra dựa trên chính nội dung của thông điệp được ký. Nếu thông điệp thay đổi dù chỉ 1 bit, chữ ký phải thay đổi hoàn toàn. Điều này ngăn chặn đòn tấn công *Existential Forgery* (giả mạo tồn tại).
- **Tính duy nhất của người gửi (Sender Uniqueness):** Chữ ký phải sử dụng thông tin duy nhất và bí mật của người gửi (Private Key). Yêu cầu này là "lá chắn" cốt lõi để chống lại việc giả mạo (forgery) từ phía người nhận và ngăn chặn việc chối bỏ trách nhiệm (denial) từ phía người gửi.
- **Tính khả thi trong tạo lập (Ease of Production):** Việc tạo ra chữ ký số phải tương đối dễ dàng và nhanh chóng đối với người gửi hợp pháp, không gây quá tải cho tài nguyên hệ thống.
- **Tính dễ dàng trong xác thực (Ease of Verification):** Mọi bên thứ ba hoặc người nhận hợp pháp đều phải có khả năng kiểm tra và xác định tính đúng đắn của chữ ký một cách thuận tiện thông qua khóa công khai.

- **Kháng giả mạo về mặt tính toán (Computational Infeasibility):** Đây là yêu cầu quan trọng nhất để đối phó với *Universal* và *Selective Forgery*. Hệ thống phải đảm bảo rằng kẻ tấn công không thể:
 - Tạo ra một thông điệp mới cho một chữ ký số đã tồn tại.
 - Tạo ra một chữ ký giả mạo cho một thông điệp đã cho sẵn.
- **Khả năng lưu trữ (Storage Practicality):** Bản sao của chữ ký số phải có kích thước tối ưu để có thể lưu trữ lâu dài nhằm phục vụ mục đích đối soát hoặc giải quyết tranh chấp pháp lý sau này.

Để thỏa mãn đồng thời các yêu cầu khắt khe này, các hệ thống hiện đại thường sử dụng một **hàm băm an toàn (Secure Hash Function)** kết hợp với các thuật toán mã hóa bất đối xứng. Hàm băm đảm bảo tính phụ thuộc nội dung và hiệu năng tính toán, trong khi mật mã bất đối xứng đảm bảo tính duy nhất và khả năng xác thực bởi bên thứ ba. Tuy nhiên, việc thiết kế chi tiết các giao thức này (như cách chèn thêm padding) cần được thực hiện cực kỳ cẩn trọng để tránh các lỗ hổng tình vi như tấn công thời gian hay tấn công bản mã lựa chọn.

2.3 Tại sao mật mã bất đối xứng chưa đủ?

Mật mã khóa công khai (Asymmetric Cryptography) đã giải quyết được một trong những bài toán nan giải nhất của ngành mật mã học: Phân phối khóa bí mật qua một kênh truyền không an toàn mà không cần hai bên phải gặp mặt trực tiếp. Bằng các đặc tính toán học ưu việt như hàm một chiều có cửa sập (Trapdoor one-way function), hệ thống này cung cấp cả hai tính năng cốt lõi là bảo mật (Confidentiality) và chữ ký số (Digital Signature).

Tuy nhiên, sức mạnh toán học đơn thuần là chưa đủ để xây dựng một hệ thống an toàn trên quy mô toàn cầu. Lỗ hổng chết người của mật mã bất đối xứng không nằm ở các phương trình toán học, mà nằm ở lỗ hổng về "Niềm tin" (Trust) trong quá trình phân phối và định danh khóa. Khi các khóa công khai thuần túy (Raw Public Keys) được truyền tải trên một mạng lưới mở như Internet, chúng trở thành miếng mồi ngon cho các đòn tấn công có chủ đích.

Tấn công Man-in-the-Middle (MITM) trên Raw Public Key

Để minh họa cho điểm yếu này, hãy xem xét kịch bản trao đổi khóa công khai nguyên thủy (Raw Public Key Exchange). Giả sử Alice muốn gửi một tài liệu mật cho Bob. Theo lý thuyết, Alice chỉ cần xin khóa công khai của Bob ($K_{\text{pub_Bob}}$) để mã hóa tài liệu. Tuy nhiên, trên một môi trường mạng bị kiểm soát bởi kẻ tấn công (Mallory), lý thuyết này hoàn toàn sụp đổ.

Quá trình tấn công Kẻ đứng giữa (Man-in-the-Middle) diễn ra âm thầm và chia cắt hoàn toàn kênh liên lạc của hai nạn nhân mà họ không hề hay biết. Mallory đóng vai trò là một trạm trung chuyển độc hại, thao túng toàn bộ luồng giao tiếp. Diễn biến chi tiết được trình bày trong Bảng 2.1.

Table 2.1: Kịch bản tấn công MITM trên quá trình trao đổi khóa công khai thuần túy

Alice	Mallory (Kẻ tấn công)	Bob
1. Gửi yêu cầu: <i>"Bob, gửi cho tôi $K_{\text{pub_Bob}}$"</i>	$\xrightarrow{\text{Đánh chặn yêu cầu}}$ Chuyển tiếp yêu cầu đến Bob.	Nhận yêu cầu từ "Alice".
	$\xleftarrow{\text{Đánh chặn } K_{\text{pub_Bob}}}$ Lưu giữ $K_{\text{pub_Bob}}$.	2. Phản hồi: Gửi $K_{\text{pub_Bob}}$ cho Alice.
3. Nhận $K_{\text{pub_Mallory}}$. Định ninh đây là khóa của Bob.	$\xrightarrow{\text{Gửi } K_{\text{pub_Mallory}}}$ Đóng giả Bob, gửi khóa công khai của chính mình cho Alice.	
4. Mã hóa tài liệu mật M : $C = \text{Enc}(K_{\text{pub_Mallory}}, M)$ Gửi C cho Bob.	$\xrightarrow{\text{Đánh chặn bản mã } C}$ Giải mã bằng khóa bí mật của Mallory: $M = \text{Dec}(K_{\text{priv_Mallory}}, C)$. (Mallory đã đọc/sửa được dữ liệu)	
	$\xrightarrow{\text{Mã hóa lại và Gửi}}$ $C' = \text{Enc}(K_{\text{pub_Bob}}, M)$ Gửi C' cho Bob.	5. Nhận C' . Giải mã bằng $K_{\text{priv_Bob}}$. Định ninh dữ liệu đến từ Alice và hoàn toàn bảo mật.

Hậu quả của đòn tấn công này là thảm họa. Mặc dù thuật toán mã hóa (ví dụ RSA) không hề bị bẻ khóa, nhưng tính bảo mật đã bị vô hiệu hóa hoàn toàn do Alice đã sử dụng "nhầm" chìa khóa để khóa cửa. Kịch bản này không chỉ đúng với việc mã hóa dữ liệu mà còn áp dụng tương tự với việc xác minh chữ ký số: Mallory có thể thay thế khóa công khai, từ đó tự do giả mạo chữ ký mang tên Bob mà Alice vẫn xác minh thành công.

Vấn đề Ràng buộc Danh tính (The Identity Binding Problem)

Cội nguồn của đòn tấn công MITM nêu trên xuất phát từ một thiếu sót nền tảng của mật mã khóa công khai: **Sự vắng mặt của tính ràng buộc danh tính (Identity Binding)**.

Về bản chất toán học, một khóa công khai (như K_{pub}) chỉ là một tập hợp các con số vô tri vô giác (ví dụ: một cặp số (e, n) trong RSA, hay một tọa độ điểm (x, y) trên đường cong Elliptic). Bản thân các con số này không chứa đựng bất kỳ siêu dữ liệu (metadata) nào để chứng minh nó thuộc về thực thể nào. Không có bất kỳ nhãn dán nào trên khóa công khai ghi rằng *"Khóa này là tài sản hợp pháp của Bob, nhân viên ngân hàng X"*.

Trong thế giới thực, để chứng minh một bức ảnh thuộc về một cái tên, chúng ta cần Chứng minh nhân dân hoặc Hộ chiếu — một tài liệu do cơ quan nhà nước có thẩm quyền cấp phép, đóng dấu giáp lai để **ràng buộc** (bind) khuôn mặt với đặc điểm nhân thân.

Trên không gian mạng, nếu Alice nhận được một chuỗi bit dài 2048-bit từ một email có tên "Bob", làm sao cô ấy có thể tin tưởng 100% đó không phải là Mallory giả mạo? Nếu không có một cơ chế kiểm chứng độc lập, mọi giao dịch dựa trên mật mã bất đối xứng đều được xây dựng trên một nền móng cát: niềm tin mù quáng vào kênh truyền tải.

Cần nổi tới Hạ tầng Khóa Công khai (PKI) Để mật mã bất đối xứng có thể vận hành trong thế giới thực, nơi mà mọi nút mạng đều có thể là kẻ thù, chúng ta bắt buộc phải có một "Sổ tư pháp" kỹ thuật số. Cần có một cơ chế, một định dạng tiêu chuẩn, và một Bên thứ ba đáng tin cậy (Trusted Third Party) đứng ra bảo lãnh cho tính chính danh của các khóa công khai.

Họ phải tạo ra một "tờ giấy thông hành" kỹ thuật số, trong đó niêm phong chặt chẽ **Khóa công khai (Public Key)** cùng với **Danh tính (Identity)** của chủ sở hữu bằng một Chữ ký số không thể làm giả. Cấu trúc đó chính là **Chứng chỉ số (Digital Certificate)**, và hệ sinh thái quản lý toàn bộ vòng đời của các chứng chỉ này chính là lý do **Hạ tầng Khóa Công khai (PKI - Public Key Infrastructure)** ra đời. Đây sẽ là trọng tâm phân tích của Chương 3.

Chapter 3

Cấu trúc hạ tầng khóa công khai PKI

Hạ tầng khóa công khai (Public Key Infrastructure - PKI) thường bị nhầm tưởng chỉ là một tập hợp các thuật toán mã hóa và phần mềm. Tuy nhiên, trên thực tế, PKI là một hệ thống toàn diện, là sự tổ hợp chặt chẽ giữa con người, quy trình, phần mềm, phần cứng và các chính sách an toàn thông tin. Mục tiêu cốt lõi của hệ thống này là tạo ra, quản lý, phân phối, sử dụng, lưu trữ và thu hồi các chứng chỉ số một cách an toàn, từ đó thiết lập và duy trì nền tảng tin cậy cho các giao dịch điện tử trên môi trường mạng không an toàn.

3.1 Các mô hình phân phối khóa công khai

Sự phát triển của các cơ chế phân phối khóa công khai trải qua nhiều giai đoạn tiến hóa, từ các phương pháp thủ công, đơn giản như Host-based Trust, phân tán như Web of Trust, cho đến các mô hình có tính hệ thống cao như Hierarchical PKI. Mỗi mô hình ra đời nhằm giải quyết những hạn chế của mô hình trước đó, nhưng đồng thời cũng phải đối mặt với những thách thức mới.

3.1.1 Bài toán phân phối khóa công khai

Trong hệ mật mã bất đối xứng, khóa công khai (Public Key) được thiết kế để công bố rộng rãi cho bất kỳ ai muốn giao tiếp an toàn với chủ sở hữu của nó. Tuy nhiên, bài toán đặt ra là làm thế nào để phân phối khóa công khai này rộng rãi mà vẫn đảm bảo tính an toàn. Quá trình phân phối này cần hướng tới việc hỗ trợ bốn mục tiêu an toàn thông tin cơ bản: tính xác thực (Authentication), tính bảo mật (Confidentiality), tính toàn vẹn (Integrity) và tính chống chối bỏ (Non-repudiation).

3.1.2 Public Announcement

Đây là phương pháp cơ bản nhất, trong đó khóa công khai được chia sẻ công khai trên một nền tảng dễ tiếp cận. Một ví dụ tiêu biểu là DNSSEC, nơi khóa công khai được lưu

trữ trực tiếp trong các bản ghi DNS và bất kỳ ai cũng có thể truy vấn thông qua các công cụ như `dig`. Tuy nhiên, điểm yếu lớn nhất của phương pháp này nằm ở khả năng xác thực (Authentication). Khi nhận được một khóa công khai, người dùng không có cơ chế đáng tin cậy nào để kiểm chứng xem khóa đó có thực sự thuộc về thực thể mà nó tự nhận hay không, dẫn đến rủi ro cao về các cuộc tấn công trung gian (Man-in-the-Middle).

3.1.3 Host-based Trust

Mô hình này dựa trên việc thiết lập sự tin cậy thủ công giữa các thiết bị. Điển hình là cơ chế của SSH, sử dụng tệp `known_hosts` để mỗi máy tính tự lưu trữ và quản lý danh sách các khóa công khai mà nó cho là an toàn. Mặc dù hiệu quả trong các hệ thống nhỏ, nhưng khi số lượng thiết bị tăng lên, số lượng kết nối tin cậy cần thiết lập sẽ tăng theo cấp số nhân ($O(n^2)$ cho n máy tính). Điều này khiến mô hình Host-based Trust hoàn toàn thiếu khả năng mở rộng (scalability) và không thể áp dụng cho mạng Internet toàn cầu.

3.1.4 Web of Trust (PGP)

Để khắc phục tính tập trung, Web of Trust (WoT) được giới thiệu trong các hệ thống như PGP (Pretty Good Privacy) [3]. Đây là một mô hình phi tập trung, dựa trên sự xác nhận lẫn nhau (peer endorsement) giữa các người dùng; mỗi cá nhân tự chịu trách nhiệm quyết định mức độ tin cậy đối với khóa của người khác dựa trên các chữ ký xác nhận của bạn bè hoặc đối tác. Dù rất dân chủ và phù hợp cho các cộng đồng kỹ thuật nhỏ, WoT gặp phải vấn đề lớn về "bootstrapping" (khó khăn cho người mới tham gia để có được sự tin cậy) và tính thân thiện với người dùng thông thường, do đó không thể mở rộng ở quy mô toàn cầu.

3.1.5 Hierarchical PKI

Đây là mô hình phổ biến và thành công nhất hiện nay trên Internet. Hierarchical PKI sử dụng cấu trúc cây phân cấp dựa trên nguyên tắc "tin cậy bắc cầu" (transitive trust). Sự tin cậy được kế thừa từ các Root CA (Tổ chức phát hành chứng chỉ gốc) trên cùng, truyền xuống các Intermediate CA, và cuối cùng đến các chứng chỉ thực thể (Leaf certificates). Ở phía máy khách, hệ điều hành và trình duyệt (như Mozilla NSS, Microsoft, Apple) duy trì một kho lưu trữ chứng chỉ gốc đáng tin cậy (Trust Store). Hoạt động của các CA này được quản lý và tiêu chuẩn hóa thông qua các yêu cầu cơ sở (baseline requirements) do CA/Browser Forum [4] ban hành.

3.1.6 So sánh trade-off

Sự khác biệt giữa các mô hình chủ yếu nằm ở cách tiếp cận quản lý sự tin cậy. Mô hình phân cấp (Hierarchical) có ưu điểm là dễ dàng kiểm toán (audit) và thu hồi chứng chỉ

(revoke), quản lý thống nhất, nhưng lại biến các Root CA thành những điểm yếu chí mạng (Single Point of Failure - SPOF) về mặt tin cậy. Ngược lại, mô hình phi tập trung như Web of Trust không có SPOF, nhưng quá trình đánh giá niềm tin lại không đủ tính bắc cầu mạnh mẽ và quá phức tạp đối với người dùng phổ thông.

Table 3.1: So sánh các mô hình tin cậy (Trust Models)

Tiêu chí	Hierarchical PKI (Centralized)	Web of Trust (Decentralized)
Cấu trúc	Cây phân cấp từ Root CA xuống Leaf	Mạng lưới đồng đẳng (Peer-to-peer)
Quản lý thu hồi	Dễ dàng, quản lý tập trung (CRL/OCSP)	Khó khăn, phụ thuộc vào bản cập nhật của từng node
Rủi ro (SPOF)	Root CA bị xâm nhập là thảm họa	Không có SPOF rõ rệt
Tính thân thiện	Rất cao (trong suốt với người dùng)	Thấp (đòi hỏi kiến thức kỹ thuật)

3.2 Định nghĩa và mục tiêu PKI

Như đã đề cập, Hạ tầng khóa công khai (PKI) là một bộ khung bao gồm các thành phần công nghệ (phần cứng, phần mềm), cơ cấu tổ chức (con người), và các tài liệu điều chỉnh (chính sách, quy trình) nhằm mục đích phát hành, duy trì, và thu hồi các chứng chỉ số số trong một môi trường mạng mở.

Mục tiêu lớn nhất của PKI là giải quyết bài toán cốt lõi về "sự tin cậy" (trust) trên môi trường số bằng cách cung cấp một cơ chế bảo đảm để gắn kết (bind) một danh tính thực (một con người, một tổ chức, hoặc một thiết bị) với một khóa công khai cụ thể thông qua bên thứ ba được tin tưởng là CA. Quan trọng hơn, PKI không chỉ dừng lại ở các giao thức kỹ thuật, mà nó mang đậm tính chất quản trị (governance) – giải quyết các vấn đề vĩ mô như: ai có đủ thẩm quyền để làm CA? Tiêu chuẩn nào CA phải tuân thủ để được đưa vào Trust Store của các trình duyệt?

****Ứng dụng của public key: Chữ ký số**** Sau khi PKI đã thiết lập sự tin cậy trong khóa công khai của một thực thể (thông qua việc cấp chứng chỉ số), thực thể đó có thể sử dụng cặp khóa của mình để đạt được các mục tiêu an toàn thông tin cụ thể, như được minh họa trong Hình 3.1. Sơ đồ này cho thấy quy trình tạo và xác thực chữ ký số, giúp đảm bảo tính toàn vẹn (Integrity) và tính xác thực (Authentication) của thông điệp được truyền đi.

Sử dụng khóa bí mật (Private Key) để ký số lên hàm băm (Message Digest) tạo ra một bằng chứng không thể phủ nhận về nguồn gốc của thông điệp. Khi người nhận dùng khóa công khai tương ứng để xác thực thành công, họ biết chắc rằng thông điệp thực sự đến từ chủ sở hữu khóa và không bị chỉnh sửa trên đường truyền. Đây chính là một trong

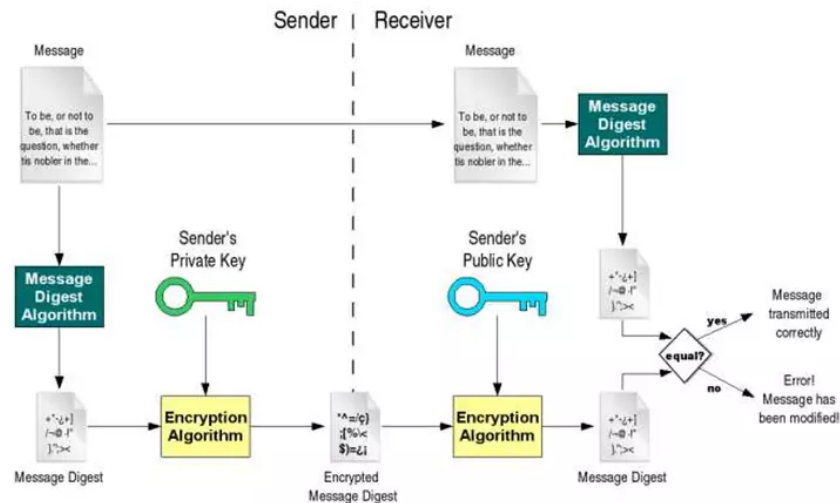


Figure 3.1: Quy trình tạo và xác thực chữ ký số sử dụng cặp khóa công khai và bí mật.

những mục tiêu hoạt động then chốt mà PKI hướng tới nhằm tạo ra sự tin cậy tổng thể.

3.3 Các thành phần PKI

CA – Certificate Authority

Certificate Authority (Tổ chức phát hành chứng chỉ) là trái tim của hệ thống PKI. CA chịu trách nhiệm chính trong việc phát hành, thu hồi chứng chỉ và duy trì danh sách chứng chỉ bị thu hồi (CRL). Để tối ưu hóa tính bảo mật, các CA thường được tổ chức theo cấu trúc phân tầng. Trên cùng là Root CA, nắm giữ khóa quan trọng nhất và thường được giữ ở trạng thái ngoại tuyến (offline, air-gapped) để tránh bị tấn công mạng. Root CA ký cấp phát chứng chỉ cho các Intermediate CA (CA trung gian), và các Intermediate CA này mới là đơn vị trực tiếp tương tác và cấp phát chứng chỉ cho thực thể cuối (Leaf/End-entity). Quy trình cơ bản bao gồm: người dùng tạo Yêu cầu ký chứng chỉ (CSR) → CA xác minh danh tính → CA dùng khóa bí mật để ký chứng chỉ → Công bố chứng chỉ.

RA – Registration Authority

Registration Authority (Tổ chức đăng ký) hoạt động như một cơ quan tiền phương cho CA. Việc tách bạch quá trình xác minh danh tính (thực hiện bởi RA) khỏi quá trình ký chứng chỉ (thực hiện bởi CA) giúp phân quyền trách nhiệm và giảm thiểu rủi ro bảo mật cho hệ thống ký. RA đóng vai trò kiểm tra tính hợp lệ của người nộp đơn. Ví dụ trong thực tế, RA có thể yêu cầu người dùng xuất trình Căn cước công dân hoặc Hộ chiếu để đối chiếu; sau khi RA xác nhận hợp lệ, yêu cầu mới được chuyển tiếp để CA thực hiện thao tác ký số sinh ra chứng chỉ.

Repository – Kho lưu trữ chứng chỉ

Kho lưu trữ (Repository) là cơ sở dữ liệu công khai lưu trữ các chứng chỉ đang có hiệu lực và danh sách các chứng chỉ đã bị thu hồi. Kho này thường được triển khai trên các máy chủ thư mục LDAP (Lightweight Directory Access Protocol) hoặc máy chủ HTTP/HTTPS, đảm bảo các client có thể truy vấn nhanh chóng để lấy chứng chỉ của người cần giao tiếp hoặc kiểm tra trạng thái CRL bất cứ lúc nào.

CRL – Certificate Revocation List

Danh sách chứng chỉ bị thu hồi (CRL) là một tài liệu được CA ký số và phát hành định kỳ, liệt kê số seri của các chứng chỉ đã bị mất hiệu lực trước khi hết hạn. Các lý do thu hồi (reason codes) thường được đi kèm, ví dụ như lộ khóa bí mật (`keyCompromise`) hoặc bản thân CA bị xâm phạm (`cACompromise`). Hạn chế lớn của CRL là kích thước tệp có thể phình to rất nhanh và thông tin không được cập nhật tức thời (có thể lỗi thời - stale trong nhiều giờ hoặc vài ngày trước khi bản mới được tải về). Để giảm tải mạng, giải pháp `delta CRL` [5] được sử dụng để chỉ tải xuống những thay đổi so với bản CRL đầy đủ trước đó.

OCSP – Online Certificate Status Protocol

Giao thức kiểm tra trạng thái chứng chỉ trực tuyến (OCSP) [6] ra đời nhằm khắc phục độ trễ của CRL. Thay vì tải toàn bộ danh sách, client gửi truy vấn chứa số seri của chứng chỉ đến máy chủ OCSP Responder và nhận về phản hồi theo thời gian thực (Realtime) về trạng thái hợp lệ của chứng chỉ đó. Để giải quyết vấn đề về quyền riêng tư (vì CA biết client đang truy cập trang web nào) và độ trễ (latency) khi client phải tự liên hệ CA, kỹ thuật **OCSP Stapling** [7] được áp dụng. Máy chủ web sẽ tự động lấy (pre-fetch) phản hồi OCSP có chữ ký của CA theo định kỳ, sau đó "gắn kèm" (staple) bằng chứng này vào quá trình thiết lập kết nối TLS ban đầu với client.

Table 3.2: So sánh các phương pháp kiểm tra trạng thái thu hồi chứng chỉ

Tiêu chí	CRL	OCSP	OCSP Stapling
Độ trễ cập nhật	Cao (phụ thuộc chu kỳ tải)	Thấp (thời gian thực)	Thấp (thời gian thực, có cache)
Chi phí băng thông	Rất cao (tải toàn bộ list)	Thấp	Rất thấp
Quyền riêng tư	Tốt (Client kiểm tra cục bộ)	Kém (CA biết lịch sử duyệt web)	Tốt (Client không gọi trực tiếp CA)

3.4 Chu kỳ sống của chứng chỉ

Hoạt động của PKI phụ thuộc vào việc quản lý chặt chẽ vòng đời (lifecycle) của một chứng chỉ số. Chu kỳ sống này bao gồm các giai đoạn tuần tự:

1. **Tạo cặp khóa (Keypair Generation):** Thực thể tự tạo Khóa công khai và Khóa bí mật một cách an toàn.
2. **Tạo yêu cầu (CSR Creation):** Gói Khóa công khai và các thông tin định danh vào một bản tin CSR (Certificate Signing Request).
3. **Xác minh (Verification):** RA/CA kiểm tra tính hợp lệ của thông tin và quyền sở hữu tên miền/tổ chức.
4. **Phát hành (Issuance):** CA ký điện tử lên CSR để tạo ra chứng chỉ số hợp lệ.
5. **Sử dụng (Usage):** Chứng chỉ được triển khai trên hệ thống (web server, email) để thực hiện mã hóa và xác thực.
6. **Gia hạn / Thu hồi (Renewal / Revocation):** Trước khi hết hạn, chứng chỉ cần được gia hạn. Nếu khóa bí mật bị lộ hoặc thông tin không còn đúng, chứng chỉ sẽ bị đưa vào quá trình thu hồi (đưa vào CRL hoặc báo trạng thái qua OCSP).

Với sự bùng nổ của quy mô dịch vụ đám mây và microservices, việc cấu hình thủ công không còn khả thi. Tự động hóa vòng đời chứng chỉ (Certificate Lifecycle Automation) thông qua các giao thức như ACME (được Let's Encrypt sử dụng phổ biến), SCEP, hoặc EST ngày càng đóng vai trò quan trọng yếu giúp đảm bảo hệ thống không bị gián đoạn dịch vụ do chứng chỉ hết hạn.

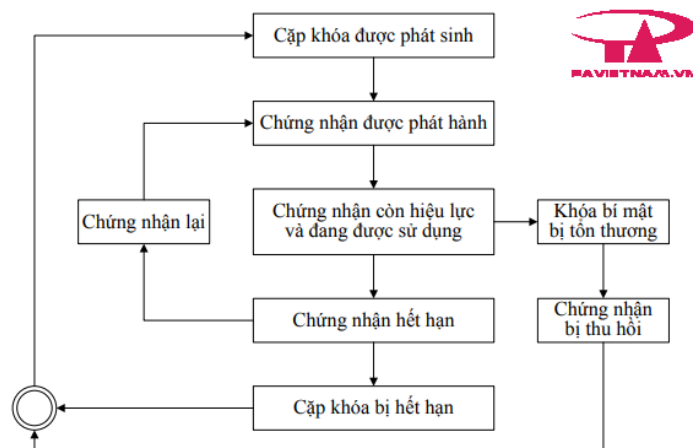


Figure 3.2: Sơ đồ chu kỳ sống của chứng chỉ số trong hệ thống PKI

Chapter 4

Chứng chỉ số và các chuẩn

Chứng chỉ số là đối tượng dữ liệu mà PKI phát hành và kiểm tra. Với chứng chỉ X.509, một CA ký lên dữ liệu gồm danh tính, khóa công khai, thời hạn hiệu lực và các ràng buộc sử dụng. Bên nhận không cần biết trước khóa công khai của chủ thể. Họ kiểm tra chữ ký của CA, chuỗi chứng chỉ, thời hạn, trạng thái thu hồi và tên được chứng nhận.

4.1 Chứng chỉ số X.509

X.509 xuất phát từ hệ thư mục X.500 của ITU-T. Trong Web PKI hiện nay, cách diễn giải được dùng rộng rãi là hồ sơ Internet X.509 do RFC 5280 định nghĩa cho chứng chỉ và CRL [5]. Phiên bản 1 chứa các trường cơ bản. Phiên bản 2 thêm định danh riêng cho issuer và subject. Phiên bản 3 bổ sung cơ chế extension, nhờ đó cùng một khuôn dạng chứng chỉ có thể biểu diễn CA, máy chủ TLS, người dùng email, thiết bị hoặc chứng chỉ ký mã.

Một chứng chỉ X.509 gồm ba phần ở mức ASN.1. Phần `tbsCertificate` là dữ liệu “to be signed”: CA ký đúng phần này. Phần thuật toán chữ ký cho biết thuật toán được dùng, chẳng hạn RSA với SHA-256 hoặc ECDSA với SHA-256. Phần giá trị chữ ký chứa chữ ký trên `tbsCertificate`. Cấu trúc này cho phép bên kiểm tra tái tạo dữ liệu đã ký và xác minh bằng khóa công khai của issuer.

X.509 được mã hóa theo DER trong tầng nhị phân. DER là một dạng mã hóa xác định của ASN.1, nên cùng một cấu trúc dữ liệu có đúng một biểu diễn byte hợp lệ. PEM là lớp bao gói văn bản: dữ liệu DER được mã hóa Base64 và đặt giữa các dòng `---BEGIN CERTIFICATE---` và `---END CERTIFICATE---`. Vì vậy, DER và PEM không phải hai loại chứng chỉ khác nhau. Chúng là hai cách biểu diễn cùng một nội dung.

Table 4.1: Các trường chính trong chứng chỉ X.509 v3

Trường	Ý nghĩa	Ghi chú
version	Phiên bản khuôn dạng chứng chỉ.	Web PKI thực tế dùng v3 để hỗ trợ extension.
serialNumber	Số seri do CA gán trong phạm vi issuer.	Được dùng khi thu hồi qua CRL hoặc OCSP.
issuer	DN của thực thể ký chứng chỉ.	Thường là CA trung gian hoặc root CA.
validity	Cặp thời điểm notBefore và notAfter .	Chứng chỉ chỉ hợp lệ trong khoảng này.
subject	DN của chủ thể được cấp chứng chỉ.	Có thể rỗng nếu danh tính nằm trong SAN.
subjectPublicKeyInfo	Thuật toán khóa công khai và giá trị khóa.	Ví dụ RSA, ECDSA hoặc EdDSA.
extensions	Các ràng buộc và thuộc tính mở rộng.	Là cơ chế trung tâm của X.509 v3.

4.2 Các trường X.509 v3

Trường **version** xác định cách diễn giải chứng chỉ. Với X.509 v3, trường **extensions** trở thành bắt buộc về mặt thực tiễn vì phần lớn kiểm tra an toàn nằm ở đây. Trường **serialNumber** phải đủ phân biệt trong phạm vi issuer để có thể định danh chứng chỉ khi thu hồi. Trường **issuer** cho biết tên của CA ký chứng chỉ; trường **subject** cho biết tên của thực thể nhận chứng chỉ.

Trường **validity** gồm **notBefore** và **notAfter**. Bên kiểm tra phải từ chối chứng chỉ nếu thời điểm hiện tại nằm ngoài khoảng này. Trường **subjectPublicKeyInfo** chứa khóa công khai của subject cùng định danh thuật toán. Với chứng chỉ máy chủ, đây là khóa được dùng để xác thực chữ ký hoặc tham gia cơ chế trao đổi khóa tùy giao thức phía trên. Chi tiết bắt tay TLS không được trình bày ở đây, vì thuộc phạm vi Chương 5.

4.3 Distinguished Name

Distinguished Name (DN) là tên phân cấp theo mô hình X.500. Trong chứng chỉ, DN thường xuất hiện ở **issuer** và **subject**. Các thuộc tính thường gặp gồm **C** cho quốc gia, **ST** cho bang hoặc tỉnh, **L** cho địa phương, **O** cho tổ chức, **OU** cho đơn vị tổ chức, **CN** cho common name và **emailAddress** cho địa chỉ thư điện tử. DN hữu ích trong môi trường tổ chức vì nó mô tả subject bằng các thuộc tính có cấu trúc.

Trong HTTPS, **CN** không còn là nơi chính để kiểm tra tên miền. RFC 6125 yêu cầu ứng dụng ưu tiên subject alternative name và không quay lại **CN** khi SAN thuộc loại tên phù hợp đã có mặt [8]. Điều này tách rõ tên hiển thị trong DN khỏi tập tên mà chứng chỉ thực sự chứng nhận. Với chứng chỉ máy chủ, SAN mới là nơi chứa DNS name hoặc IP

address được so khớp với tên mà client truy cập.

4.4 Extension quan trọng

Extension làm cho X.509 v3 phù hợp với nhiều ngữ cảnh sử dụng. Mỗi extension có định danh OID, giá trị và cờ critical. Nếu một extension critical không được hiểu, bên kiểm tra phải từ chối chứng chỉ theo RFC 5280 [5]. Cơ chế này cho phép CA áp đặt điều kiện an toàn mà client không được bỏ qua.

Table 4.2: Một số extension thường gặp trong X.509 v3

Extension	Nội dung	Vai trò
BasicConstraints	Cờ cA và tùy chọn pathLenConstraint.	Phân biệt CA với end-entity certificate.
KeyUsage	Bit như digitalSignature, keyEncipherment, keyCertSign.	Giới hạn thao tác mật mã được phép.
ExtendedKeyUsage	OID mục đích như server authentication, client authentication hoặc code signing.	Thu hẹp ngữ cảnh sử dụng của khóa.
SubjectAltName	DNS name, IP address, email hoặc URI của subject.	Cơ sở kiểm tra tên miền trong Web PKI.
SubjectKeyIdentifier	Định danh suy ra từ khóa công khai của subject.	Hỗ trợ tìm và nối chuỗi chứng chỉ.
AuthorityKeyIdentifier	Định danh khóa của issuer.	Ghép chứng chỉ với CA đã ký nó.
CRLDistributionPoints	Vị trí tải CRL.	Cho phép client kiểm tra thu hồi bằng CRL.
AuthorityInfoAccess	Vị trí OCSP responder hoặc chứng chỉ issuer.	Hỗ trợ kiểm tra trạng thái và dựng chuỗi.

BasicConstraints là extension bắt buộc về mặt an toàn với CA certificate. Nếu cA bằng TRUE, chứng chỉ có thể được dùng để ký chứng chỉ khác, với điều kiện các trường khác như **KeyUsage** cũng cho phép. **pathLenConstraint** giới hạn số CA trung gian có thể xuất hiện bên dưới CA đó trong chuỗi chứng chỉ.

KeyUsage mô tả thao tác mật mã cấp thấp. Ví dụ, **digitalSignature** cho phép dùng khóa để kiểm tra chữ ký số, trong khi **keyCertSign** chỉ phù hợp với CA. Ngược lại, **ExtendedKeyUsage** mô tả mục đích ứng dụng. Một chứng chỉ có thể chứa khóa hợp lệ về mặt thuật toán nhưng vẫn bị từ chối nếu ECU không chứa mục đích mà ứng dụng cần.

SubjectAltName chứa các tên được chứng nhận cho subject. Với chứng chỉ website, SAN thường chứa một hoặc nhiều DNS name. **SubjectKeyIdentifier** và **AuthorityKeyIdentifier** không tự tạo ra niềm tin, nhưng giúp phần mềm chọn đúng issuer khi có nhiều chứng chỉ CA cùng tên hoặc cùng tổ chức. Hai extension **CRLDistributionPoints** và **AuthorityInfoAccess** liên kết chứng chỉ với hạ tầng kiểm tra thu hồi qua CRL và OCSP [6].

4.5 Certificate Signing Request

Certificate Signing Request (CSR) là yêu cầu ký chứng chỉ do chủ thể gửi cho CA. Khuôn dạng CSR phổ biến là PKCS #10, được RFC 2986 mô tả [9]. CSR chứa thông tin subject, khóa công khai, một tập thuộc tính tùy chọn và chữ ký do chính khóa bí mật ứng với khóa công khai trong CSR tạo ra. Chữ ký này chứng minh rằng người gửi CSR có quyền kiểm soát khóa bí mật, nhưng nó không chứng minh danh tính pháp lý hoặc quyền sở hữu tên miền. Phần đó thuộc trách nhiệm xác minh của RA hoặc CA.

Luồng cấp chứng chỉ có bốn bước. Thứ nhất, subject tạo cặp khóa và bảo vệ khóa bí mật cục bộ. Thứ hai, subject tạo CSR chứa khóa công khai và thông tin định danh mong muốn. Thứ ba, CA kiểm tra quyền kiểm soát tên miền, thông tin tổ chức hoặc tiêu chí tương ứng với loại chứng chỉ. Thứ tư, CA ký dữ liệu X.509 và trả lại chứng chỉ. Khóa bí mật không cần rời khỏi hệ thống của subject trong luồng thông thường.

4.6 Phân loại chứng chỉ

CA/Browser Forum Baseline Requirements định nghĩa yêu cầu tối thiểu cho chứng chỉ máy chủ TLS được tin cậy công khai, bao gồm quy trình xác minh tên miền, thông tin subject và vận hành CA [4]. Trong thực tế, chứng chỉ TLS thường được phân loại theo mức xác minh danh tính: domain validation (DV), organization validation (OV) và extended validation (EV). DV xác minh quyền kiểm soát tên miền. OV thêm kiểm tra tổ chức. EV áp dụng bộ yêu cầu nhận diện tổ chức chặt hơn theo tài liệu EV riêng của CA/Browser Forum [10].

Table 4.3: Phân loại chứng chỉ theo mục đích sử dụng

Loại	Đặc điểm	Cách dùng
DV	Xác minh quyền kiểm soát tên miền.	Website công khai, tự động hóa cấp phát.
OV	Xác minh thêm thông tin tổ chức.	Dịch vụ cần hiện thông tin pháp nhân rõ hơn.
EV	Xác minh tổ chức theo yêu cầu EV.	Môi trường cần quy trình nhận diện chặt hơn.
Wildcard	Tên dạng *.example.com.	Nhiều subdomain cùng một cấp.
Multi-domain/SAN	Nhiều tên trong SubjectAltName.	Gom nhiều DNS name vào một chứng chỉ.
Code signing	EKU cho ký mã hoặc chính sách tương ứng.	Ký phần mềm, script hoặc gói cài đặt.
S/MIME	Gắn khóa công khai với địa chỉ email.	Ký và mã hóa thư điện tử [11].
Client certificate	Subject đại diện người dùng, thiết bị hoặc dịch vụ.	Xác thực client trong VPN, mTLS hoặc hệ nội bộ.

Wildcard certificate chỉ bao phủ một cấp nhân theo cách diễn giải thông thường của Web PKI. Ví dụ, *.example.com có thể khớp www.example.com, nhưng không khớp a.b.example.com. Multi-domain certificate dùng SAN để chứa nhiều tên cụ thể. Cách này phù hợp khi một dịch vụ có nhiều tên miền hoặc khi cần hợp nhất chứng chỉ cho các endpoint liên quan.

4.7 Các chuẩn PKCS

PKCS là họ chuẩn ban đầu do RSA Laboratories công bố cho các đối tượng mật mã trong triển khai thực tế. Trong phạm vi chứng chỉ số, ba chuẩn cần quan tâm là PKCS #7/CMS, PKCS #10 và PKCS #12. Báo cáo không liệt kê toàn bộ họ PKCS vì nhiều chuẩn đã lỗi thời hoặc nằm ngoài trọng tâm PKI.

PKCS #7 định nghĩa cú pháp cho dữ liệu được ký hoặc mã hóa. Hậu thân chuẩn hóa trong IETF là Cryptographic Message Syntax (CMS), được RFC 5652 mô tả [12]. CMS là nền tảng cho nhiều định dạng chữ ký và thông điệp bảo mật, trong đó có S/MIME. PKCS #10 là định dạng CSR như đã trình bày ở Mục 4.5. PKCS #12 định nghĩa định dạng đóng gói khóa bí mật, chứng chỉ và chuỗi chứng chỉ vào một tệp, thường có phần bảo vệ bằng mật khẩu; RFC 7292 mô tả định dạng này [13]. Tệp .p12 hoặc .pfx thường được dùng khi xuất nhập chứng chỉ người dùng hoặc chứng chỉ máy chủ giữa các hệ thống.

4.8 RFC và chuẩn quan trọng

RFC 5280 là chuẩn nền tảng cho hồ sơ Internet X.509, bao gồm cấu trúc chứng chỉ, extension, thuật toán dựng đường chứng chỉ và CRL [5]. RFC 2986 định nghĩa PKCS #10 cho CSR [9]. RFC 5652 định nghĩa CMS, là khuôn dạng chung cho dữ liệu ký số và mã hóa ở tầng thông điệp [12]. RFC 7292 định nghĩa PKCS #12 cho gói khóa và chứng chỉ [13].

RFC 6960 định nghĩa OCSP, giao thức hỏi trạng thái thu hồi cho một chứng chỉ cụ thể [6]. RFC 6962 định nghĩa Certificate Transparency phiên bản thử nghiệm, trong đó chứng chỉ TLS được ghi vào log công khai dựa trên cây Merkle [14]. Các triển khai mới hơn có thể tham chiếu RFC 9162 cho Certificate Transparency phiên bản 2.0 [15]. CA/Browser Forum Baseline Requirements không thay thế RFC, nhưng đặt yêu cầu vận hành cho CA muốn được tin cậy trong hệ sinh thái trình duyệt [4]. Vì vậy, Web PKI là kết quả của cả chuẩn kỹ thuật và quy tắc quản trị CA.

Chapter 5

Triển khai và ứng dụng thực tế

PKI đi vào hệ thống thực tế bằng một mẫu chung. Một thực thể cần chứng minh danh tính hoặc quyền phát hành. Thực thể đó dùng chứng chỉ để gắn danh tính với khóa công khai. Bên nhận kiểm tra chuỗi chứng chỉ, mục đích sử dụng khóa và trạng thái hiệu lực trước khi tin vào chữ ký hoặc khóa phiên.

Chương này không lặp lại cấu trúc X.509 đã trình bày ở Chương 4. Trọng tâm là vai trò vận hành của PKI trong HTTPS, ký số tài liệu, S/MIME, VPN, ký mã và dịch vụ công điện tử.

Table 5.1: So sánh một số ứng dụng PKI

Ứng dụng	Đối tượng xác thực	Loại chứng chỉ	Kiểm tra chính	Rủi ro vận hành
HTTPS/TLS	Máy chủ dịch vụ	TLS server certificate	Chuỗi CA, SAN, thời hạn, KU/EKU, thu hồi nếu có	Sai hostname, thiếu intermediate CA, khóa máy chủ bị lộ.
Ký số tài liệu	Người ký hoặc tổ chức	Chứng chỉ ký số cá nhân, tổ chức hoặc dịch vụ ký từ xa	Chữ ký tài liệu, chuỗi CA, thời điểm ký, trạng thái thu hồi	Không có timestamp tin cậy, khóa ký bị dùng ngoài kiểm soát.
S/MIME	Người gửi và người nhận email	Chứng chỉ gắn với địa chỉ email	Chữ ký email, chứng chỉ người gửi, chứng chỉ người nhận khi mã hóa	Khó quản lý chứng chỉ người dùng, mất khóa giải mã.
VPN/IPSec	Gateway, client hoặc site ngang hàng	Chứng chỉ thiết bị hoặc người dùng	CERT, CERTREQ, AUTH trong IKEv2, chuỗi CA nội bộ	CA nội bộ yếu, cấp chứng chỉ sai danh tính, thu hồi chậm.
Code signing	Nhà phát hành phần mềm	Code signing certificate	Chữ ký mã, certificate chain, timestamp, trạng thái thu hồi	Private key bị lộ, pipeline build bị chiếm quyền.
eGovernment	Công dân, tổ chức, cơ quan, dịch vụ ký số	Chứng chỉ công cộng hoặc chứng chỉ ký số từ xa	Chữ ký văn bản, chứng chỉ người ký, timestamp, OCSP/CRL	Liên thông hệ thống, vòng đời chứng chỉ, trải nghiệm người dùng.

5.1 HTTPS / TLS

Bài toán thực tế. Client cần biết mình đang nói chuyện với đúng máy chủ. Nếu chỉ có mã hóa kênh truyền, kẻ trung gian vẫn có thể đưa khóa công khai giả. Trong Web PKI,

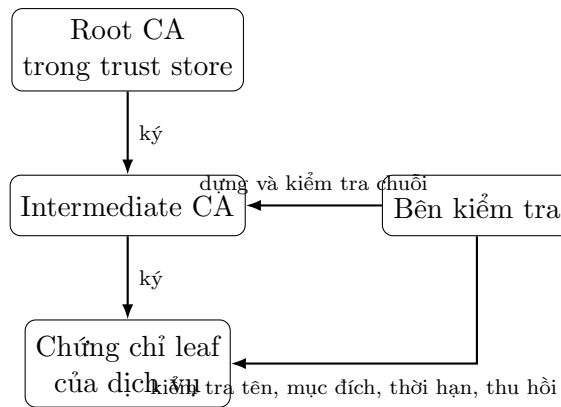


Figure 5.1: Mẫu kiểm tra PKI trong ứng dụng thực tế

chứng chỉ máy chủ TLS giải quyết bài toán gắn khóa công khai với DNS name mà người dùng truy cập.

Cách PKI giải quyết. Trong TLS 1.3, máy chủ thường gửi thông điệp **Certificate** chứa chứng chỉ leaf và các chứng chỉ trung gian cần thiết. Máy chủ sau đó dùng khóa bí mật tương ứng để tạo **CertificateVerify**, qua đó chứng minh quyền kiểm soát khóa trong chứng chỉ [16]. Root CA không được gửi như một bằng chứng mới. Nó phải có sẵn trong trust store của hệ điều hành hoặc trình duyệt.

Certificate chain nối từ leaf đến intermediate CA, rồi đến root CA được tin cậy. Leaf certificate chứng nhận tên miền dịch vụ. Intermediate CA giảm rủi ro cho root CA vì root có thể được giữ offline. Root CA là điểm neo tin cậy mà client dùng để kết thúc quá trình dựng chuỗi.

Luồng kiểm tra. Client kiểm tra chữ ký trên từng mắt xích của chuỗi. Nó từ chối chứng chỉ nếu thời điểm hiện tại nằm ngoài khoảng hiệu lực. Tên truy cập phải khớp với DNS name hoặc IP address trong SAN, theo nguyên tắc đã nêu ở Mục 4.3. Client cũng kiểm tra **KeyUsage** và **ExtendedKeyUsage** để bảo đảm chứng chỉ phù hợp với xác thực máy chủ.

Kiểm tra thu hồi có thể dùng CRL, OCSP hoặc OCSP stapling. OCSP được RFC 6960 định nghĩa cho truy vấn trạng thái của một chứng chỉ cụ thể [6]. OCSP stapling cho phép máy chủ gửi kèm phản hồi OCSP trong TLS, thay vì để client tự hỏi responder của CA [7]. Certificate Transparency bổ sung khả năng giám sát cấp phát chứng chỉ TLS bằng log công khai. RFC 9162 mô tả CT như một cơ chế ghi nhận chứng chỉ và precertificate trong log có thể kiểm toán [15].

ACME tự động hóa quá trình yêu cầu, xác minh quyền kiểm soát tên miền, cấp, gia hạn và thu hồi chứng chỉ. RFC 8555 định nghĩa giao thức này, thường được biết đến qua hệ sinh thái Let's Encrypt [17]. ACME không thay thế kiểm tra PKI ở phía client. Nó làm giảm lỗi vận hành ở phía cấp và gia hạn chứng chỉ.

Lỗi và rủi ro triển khai. Lỗi phổ biến là thiếu intermediate CA trong cấu hình máy chủ. Khi đó một số client không dựng được chuỗi, dù leaf certificate hợp lệ. Lỗi khác là chứng chỉ không chứa đúng SAN, hết hạn, dùng sai EKU, hoặc private key bị sao chép khỏi máy chủ. Trong môi trường nhiều dịch vụ, tự động hóa gia hạn phải đi kèm giám sát. Một chứng chỉ hết hạn có thể làm ngừng dịch vụ dù hệ thống ứng dụng vẫn hoạt động.

5.2 Chữ ký số văn bản

Bài toán thực tế. Văn bản điện tử cần cơ chế kiểm tra người ký và phát hiện sửa đổi sau khi ký. Việc chèn hình ảnh chữ ký không cung cấp bảo đảm mật mã. Nó không chứng minh người ký kiểm soát khóa bí mật, và cũng không ràng buộc toàn vẹn tài liệu.

Cách PKI giải quyết. Luồng ký số bắt đầu bằng việc tính giá trị băm của tài liệu hoặc cấu trúc dữ liệu cần ký. Phần mềm ký dùng private key để ký giá trị đó. Chữ ký, chứng chỉ người ký và có thể cả chuỗi chứng chỉ được gắn vào tài liệu hoặc gói chữ ký. Người nhận dùng public key trong chứng chỉ để kiểm tra chữ ký, rồi kiểm tra chuỗi CA và trạng thái chứng chỉ.

Chữ ký số, chứng chỉ và timestamp có vai trò khác nhau. Chữ ký số ràng buộc tài liệu với private key. Chứng chỉ ràng buộc public key với danh tính người ký hoặc tổ chức. Timestamp ghi nhận rằng chữ ký đã tồn tại tại một thời điểm do Time Stamping Authority xác nhận. RFC 3161 định nghĩa Time-Stamp Protocol cho hạ tầng X.509 [18].

Luồng kiểm tra. Bên nhận kiểm tra chữ ký trên tài liệu trước. Sau đó họ kiểm tra chứng chỉ người ký, bao gồm chain, thời hạn, mục đích sử dụng khóa và trạng thái thu hồi. Nếu tài liệu có timestamp đáng tin cậy, bên nhận có thể đánh giá chữ ký tại thời điểm ký. Điều này giúp kiểm chứng tài liệu sau khi chứng chỉ người ký đã hết hạn, miễn là chứng chỉ còn hợp lệ tại thời điểm được đóng dấu.

Với tài liệu PDF, chữ ký số thường được chuẩn hóa theo các profile riêng của hệ sinh thái tài liệu. Báo cáo này chỉ dùng khái niệm ở mức PKI: tài liệu được ký, chứng chỉ được đính kèm, và timestamp giúp bảo vệ giá trị xác minh theo thời gian. Phần chi tiết định dạng PDF không phải trọng tâm của chương.

Lỗi và rủi ro triển khai. Rủi ro chính là bảo vệ private key. Nếu khóa ký nằm trên máy người dùng mà không có thiết bị bảo vệ hoặc chính sách truy cập phù hợp, chữ ký có thể bị tạo ngoài ý chí người ký. Timestamp cũng cần nguồn tin cậy. Nếu tài liệu chỉ có thời gian hệ thống cục bộ, verifier khó phân biệt chữ ký hợp lệ trước khi hết hạn với chữ ký được tạo muộn sau đó.

5.3 S/MIME

Bài toán thực tế. Email mặc định không cung cấp xác thực mạnh cho nội dung từng thư. Người nhận cần biết thư có thật do người gửi tạo ra không. Khi cần bí mật nội dung, người gửi cũng cần mã hóa thư cho đúng người nhận.

Cách PKI giải quyết. S/MIME dùng chứng chỉ X.509 gắn với địa chỉ email. RFC 8551 định nghĩa S/MIME 4.0 cho ký và mã hóa dữ liệu MIME [11]. Khi ký email, người gửi dùng private key của mình để tạo chữ ký. Người nhận dùng chứng chỉ của người gửi để kiểm tra chữ ký và chuỗi CA.

Khi mã hóa email, người gửi cần chứng chỉ của từng người nhận. Nội dung thư thường được mã hóa bằng khóa phiên. Khóa phiên đó được bảo vệ bằng public key của người nhận. Chỉ người có private key tương ứng mới giải mã được nội dung.

Luồng kiểm tra. Với thư ký số, client email kiểm tra chữ ký trên nội dung, sau đó kiểm tra chứng chỉ người gửi. Với thư mã hóa, người nhận dùng private key của mình để mở khóa phiên, rồi giải mã nội dung. Nếu thư vừa ký vừa mã hóa, verifier cần xử lý cả hai lớp. Thứ tự xử lý phụ thuộc vào định dạng thông điệp cụ thể.

S/MIME khác PGP ở mô hình tin cậy. S/MIME dựa vào CA và chứng chỉ X.509. PGP thường gắn với Web of Trust hoặc cơ chế tự quản lý khóa. Điều này làm S/MIME phù hợp hơn với tổ chức có PKI tập trung, nhưng phụ thuộc vào cấp phát và quản trị chứng chỉ người dùng.

Lỗi và rủi ro triển khai. Vấn đề thực tế lớn nhất là vòng đời chứng chỉ người dùng. Người dùng có thể đổi máy, mất private key, hoặc dùng nhiều client email khác nhau. Nếu private key mã hóa bị mất, thư cũ có thể không đọc lại được. Nếu tổ chức cấp chứng chỉ cho nhiều địa chỉ hoặc đổi địa chỉ email, chính sách ánh xạ danh tính cần rõ ràng.

5.4 VPN và IPSec

Bài toán thực tế. VPN cần xác thực hai đầu kết nối trước khi thiết lập kênh bảo vệ. Trong mô hình site-to-site, hai gateway cần nhận ra nhau. Trong mô hình remote access, gateway cần xác thực client hoặc người dùng. Nếu chỉ dùng một pre-shared key chung, việc mở rộng và thu hồi quyền truy cập trở nên khó kiểm soát.

Cách PKI giải quyết. IKEv2 thiết lập Security Association cho IPSec. RFC 7296 mô tả việc các peer có thể dùng payload CERT, CERTREQ và AUTH trong trao đổi IKE_AUTH [19]. Chứng chỉ cung cấp public key và danh tính. Payload AUTH chứng minh peer sở hữu private key tương ứng.

Tổ chức thường dùng CA nội bộ cho VPN. CA cấp chứng chỉ cho gateway, thiết bị hoặc người dùng. Gateway tin root CA nội bộ và kiểm tra chứng chỉ client theo chính sách truy cập. Hai gateway site-to-site có thể xác thực lẫn nhau bằng chứng chỉ do cùng một CA hoặc các CA được liên thông cấp.

Luồng kiểm tra. Peer gửi danh tính, chứng chỉ và bằng chứng xác thực. Bên nhận dựng chuỗi tới CA được tin cậy, kiểm tra thời hạn và trạng thái thu hồi, rồi kiểm tra rằng danh tính trong chứng chỉ phù hợp với chính sách VPN. Với mutual authentication, cả hai phía đều thực hiện quy trình này. Sau khi xác thực thành công, IKEv2 tạo các Child SA để bảo vệ lưu lượng ESP hoặc AH.

Lỗi và rủi ro triển khai. Certificate-based VPN mở rộng tốt hơn pre-shared key vì mỗi thiết bị hoặc người dùng có thể có chứng chỉ riêng. Khi một client mất quyền, CA có thể thu hồi chứng chỉ đó mà không đổi bí mật của toàn hệ thống. Đổi lại, tổ chức phải vận hành PKI nghiêm túc. Lỗi cấu hình CA, bỏ qua thu hồi, cấp chứng chỉ sai identity hoặc lưu private key gateway không an toàn đều làm giảm lợi ích của mô hình này.

5.5 Code Signing

Bài toán thực tế. Người dùng và hệ điều hành cần biết phần mềm đến từ nhà phát hành nào và có bị sửa sau khi ký không. Kênh tải xuống không đủ để bảo đảm hai điều này. Một file có thể bị thay đổi trên mirror, trong quá trình phân phối, hoặc trong pipeline phát hành.

Cách PKI giải quyết. Code signing gắn chữ ký số vào phần mềm, script, driver hoặc gói cài đặt. Microsoft mô tả chữ ký số trên file như một cách nhận diện nhà phân phối và bảo đảm nội dung không đổi sau khi chữ ký được tạo [20]. Apple cũng mô tả code signing như cơ chế xác minh tính toàn vẹn ứng dụng và danh tính nhà phát triển [21].

Chứng chỉ ký mã khác chứng chỉ TLS ở mục đích sử dụng. Nó dùng để chứng minh DNS name của máy chủ. Nó dùng cho publisher hoặc tổ chức phát hành phần mềm. Sự khác biệt này thường được thể hiện bằng EKU, chính sách chứng chỉ và quy tắc của nền tảng kiểm tra.

Luồng kiểm tra. Khi cài hoặc chạy phần mềm, hệ thống kiểm tra chữ ký trên file. Nếu chữ ký hợp lệ, hệ thống dựng chuỗi chứng chỉ của publisher tới một root được tin cậy. Nó cũng có thể kiểm tra timestamp và trạng thái thu hồi. Timestamp giúp phân biệt phần mềm đã được ký khi chứng chỉ còn hợp lệ với phần mềm được ký sau khi chứng chỉ hết hạn hoặc bị thu hồi.

Lỗi và rủi ro triển khai. Rủi ro nghiêm trọng nhất là private key ký mã bị lộ. Khi đó kẻ tấn công có thể ký mã độc dưới danh tính hợp lệ cho tới khi chúng chỉ bị thu hồi và các nền tảng cập nhật trạng thái. Một rủi ro khác là build pipeline bị compromise. Trong trường hợp đó, phần mềm độc hại có thể được ký bằng khóa thật của tổ chức. Vì vậy, code signing cần kiểm soát khóa, kiểm soát pipeline và quy trình thu hồi rõ ràng.

5.6 eGovernment Việt Nam

Bài toán thực tế. Dịch vụ công điện tử cần xác thực công dân, tổ chức, cán bộ xử lý và văn bản điện tử. Hệ thống cũng cần kiểm tra văn bản đã ký sau khi nó được gửi qua nhiều cổng dịch vụ hoặc lưu trữ lâu dài. Bài toán chính không chỉ là tạo chữ ký, mà là quản lý niềm tin ở quy mô quốc gia.

Cách PKI giải quyết. Trang thông tin chính thức của Trung tâm Chứng thực điện tử quốc gia (NEAC) thể hiện các chức năng liên quan đến CA công cộng, ký số tài liệu, ký số từ xa và kiểm tra văn bản ký số [22]. Ở mức vai trò hệ thống, CA cấp chứng chỉ cho cá nhân, tổ chức hoặc dịch vụ. Người dùng dùng chứng chỉ để ký hồ sơ, văn bản hoặc giao dịch. Dịch vụ công kiểm tra chữ ký và chuỗi chứng chỉ trước khi chấp nhận dữ liệu.

Ký số từ xa chuyển phần kiểm soát khóa sang dịch vụ được vận hành chuyên biệt. Người dùng xác thực với dịch vụ ký, yêu cầu ký tài liệu, rồi nhận chữ ký gắn với chứng chỉ của mình. Mô hình này giảm gánh nặng cài đặt thiết bị ký ở phía người dùng, nhưng đặt yêu cầu cao hơn lên xác thực người dùng, nhật ký ký và bảo vệ hệ thống ký.

Luồng kiểm tra. Một cổng dịch vụ công hoặc hệ thống văn thư nhận tài liệu đã ký. Nó kiểm tra chữ ký trên tài liệu, chứng chỉ người ký, chain đến CA được tin cậy, thời điểm ký và trạng thái thu hồi. Trang NEAC cũng thể hiện chức năng kiểm tra văn bản PDF/XML, kèm thông tin hiệu lực chứng chỉ, chữ ký, timestamp, OCSP và CRL [22]. Những trường này phản ánh các thành phần mà verifier cần khi đánh giá văn bản ký số.

Lỗi và rủi ro triển khai. Rủi ro chính là tính liên thông. Các hệ thống khác nhau phải hiểu đúng định dạng chữ ký, chính sách chứng chỉ, timestamp và trạng thái thu hồi. Nếu verifier chỉ kiểm tra chữ ký mà bỏ qua chain hoặc thu hồi, hệ thống có thể chấp nhận chữ ký từ chứng chỉ không còn hợp lệ. Nếu quy trình ký từ xa xác thực người dùng yếu, chữ ký hợp lệ về mặt mật mã vẫn có thể không phản ánh đúng ý chí của chủ thể.

Chapter 6

Các hệ thống PKI mã nguồn mở

PKI trong triển khai thực tế không chỉ là một chuẩn chứng chỉ. Nó là một hệ sinh thái công cụ. Mỗi công cụ giải quyết một phần khác nhau của bài toán: thao tác mật mã, quản lý CA cục bộ, tự động hóa TLS công cộng, phát hành chứng chỉ nội bộ, hoặc tích hợp với hệ thống định danh.

Chương này khảo sát các hệ thống và công cụ mã nguồn mở theo vai trò công nghệ. Trọng tâm là vị trí của từng công cụ trong hệ sinh thái PKI, không phải quy trình demo hoặc thiết kế triển khai của nhóm.

6.1 Tiêu chí phân loại

Một công cụ PKI có thể được phân loại theo vai trò chính. Nhóm thứ nhất là *cryptographic toolkit*, gồm thư viện và CLI để tạo khóa, tạo CSR, ký, chuyển đổi và kiểm tra chứng chỉ. OpenSSL thuộc nhóm này.

Nhóm thứ hai là công cụ quản lý chứng chỉ cục bộ. Chúng phù hợp với lab, học tập hoặc hệ thống nhỏ cần giao diện trực quan. XCA là ví dụ tiêu biểu vì nó quản lý khóa, CSR, chứng chỉ, CRL và template trong cơ sở dữ liệu cục bộ [23].

Nhóm thứ ba là ACME client và public CA tự động hóa. Let's Encrypt là CA công cộng miễn phí và tự động. Certbot là ACME client dùng để yêu cầu, cài đặt và gia hạn chứng chỉ từ các CA tương thích ACME [17], [24], [25].

Nhóm thứ tư là CA platform. Đây là các hệ thống quản trị PKI có profile, quy trình đăng ký, phát hành, thu hồi, API và tích hợp vận hành. EJBCA và Dogtag nằm trong nhóm này [26], [27].

Nhóm thứ năm là internal PKI hoặc secret management. Vault PKI Secrets Engine phát hành chứng chỉ X.509 nội bộ qua API và chính sách truy cập, thường dùng cho dịch vụ nội bộ hoặc mTLS [28].

Nhóm cuối là identity và authentication stack. OAuth 2.0 và OpenID Connect không phải nền tảng PKI. Chúng dùng TLS, chữ ký số, JWT, JWS và JWKS trong hệ xác thực và phân quyền [29], [30], [31], [32], [33].

Các tiêu chí so sánh gồm chức năng CA, giao diện, API, ACME, vòng đời chứng chỉ, thu hồi, OCSP/CRL, audit, tích hợp định danh và độ phức tạp vận hành. Một công cụ tốt trong lab chưa chắc phù hợp để vận hành CA có chính sách. Ngược lại, nền tảng enterprise thường quá nặng cho nhu cầu kiểm thử cục bộ.

6.2 OpenSSL

OpenSSL là thư viện và dòng lệnh nền tảng cho nhiều thao tác mật mã. Trong bối cảnh PKI, hai lệnh quan trọng là `openssl x509` và `openssl verify`. Tài liệu OpenSSL mô tả `x509` là lệnh hiển thị và ký chứng chỉ. Nó có thể xem trường chứng chỉ, chuyển đổi PEM và DER, tạo chứng chỉ từ PKCS #10 request, tự ký chứng chỉ và ký bằng CA key [34].

Lệnh `verify` kiểm tra chuỗi chứng chỉ. Nó nhận trust anchor qua `-CAfile`, `-CApath` hoặc nguồn tin cậy tương ứng. Nó cũng có tùy chọn kiểm tra hostname, email, IP và mục đích sử dụng chứng chỉ [35]. Vì vậy, OpenSSL là công cụ cơ sở để quan sát và kiểm tra hành vi PKI.

Một số lệnh ngắn cho thấy vai trò công nghệ của OpenSSL:

```
1 openssl x509 -in server.crt -text -noout
2 openssl verify -CAfile ca-chain.pem server.crt
3 openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -out
  server.crt
```

Listing 6.1: Một số thao tác PKI với OpenSSL

Giới hạn của OpenSSL nằm ở tầng quản trị. Nó không tự cung cấp portal RA, profile người dùng, phê duyệt, audit tập trung, HSM policy hoặc API vòng đời chứng chỉ. Có thể xây dựng CA đơn giản bằng OpenSSL, nhưng phần quản trị sẽ nằm trong script và quy trình bên ngoài.

6.3 XCA

XCA là công cụ giao diện đồ họa để xây dựng và quản trị các đối tượng PKI dựa trên X.509. Trang chính thức mô tả XCA quản lý chứng chỉ, CSR, khóa riêng RSA/DSA/EC, smartcard và CRL. Dữ liệu được lưu trong cơ sở dữ liệu như SQLite, MySQL, PostgreSQL hoặc SQL Server qua ODBC [23].

XCA phù hợp với môi trường local và lab. Người dùng có thể tạo CA nhỏ, tạo sub-CA, dùng template cho chứng chỉ và xem chuỗi chứng chỉ trong giao diện. Điều này làm XCA dễ tiếp cận hơn OpenSSL khi mục tiêu là học cấu trúc PKI hoặc quản lý một kho chứng chỉ nhỏ.

So với OpenSSL, XCA mạnh ở trực quan hóa và quản lý local certificate store. OpenSSL mạnh hơn ở CLI, scripting và kiểm tra tự động trong pipeline. Cả hai đều không nên

được mô tả như CA platform enterprise. Chúng thiếu các lớp vận hành như RA workflow, phân quyền quản trị, API chuẩn hóa và audit tập trung.

6.4 EJBCA

EJBCA là trọng tâm của nhóm CA platform mã nguồn mở. Tài liệu chính thức mô tả EJBCA như hệ thống có hướng dẫn cho CA, RA, vận hành, tích hợp và khắc phục sự cố [26]. So với OpenSSL hoặc XCA, EJBCA không chỉ ký chứng chỉ. Nó quản lý chính sách phát hành và vòng đời chứng chỉ.

Trong EJBCA, certificate profile là template giới hạn cấu trúc chứng chỉ. Profile có thể quy định thuật toán khóa, kích thước khóa, mục đích chứng chỉ, Key Usage, Extended Key Usage, Certificate Transparency và CA được phép phát hành [36]. End entity profile là template cho subject DN và thông tin của chủ thể. Nó cũng giới hạn certificate profile mà một end entity được dùng khi đăng ký [37].

Cách tách certificate profile khỏi end entity profile cho phép EJBCA mô hình hóa chính sách CA rõ hơn công cụ local. Certificate profile mô tả loại chứng chỉ được phát hành. End entity profile mô tả đối tượng được đăng ký và quyền dùng profile. RA hoặc quy trình enrollment có thể dựa vào hai lớp này để kiểm soát yêu cầu cấp chứng chỉ.

EJBCA có REST interface cho quản trị CA và quản lý chứng chỉ. Tài liệu REST nêu các nhóm API cho ACME, CA management, certificate, end entity, approval, crypto token và ConfigDump. REST API mặc định bị tắt và có thể dùng xác thực bằng client certificate hoặc OAuth [38].

EJBCA cũng hỗ trợ ACME. Tài liệu ACME của EJBCA ghi nhận việc triển khai RFC 8555, hỗ trợ order, authorization, challenge, finalize, revocation, key change, tải chứng chỉ và renewal information. Các challenge được nêu gồm `http-01`, `dns-01`, `tls-alpn-01` và một số kiểu mở rộng [39].

Về thu hồi, EJBCA cung cấp OCSP responder. Tài liệu OCSP mô tả endpoint `/ejbca/publicweb/status/ocsp`, OCSP Key Bindings, cấu hình AIA trong certificate profile và transaction logs cho request/response [40]. Những chức năng này đặt EJBCA gần với môi trường CA vận hành thực tế hơn công cụ chỉ tạo chứng chỉ.

Tài liệu EJBCA cũng có phần tích hợp Hardware Security Modules trong mục integration [26]. Điều này quan trọng vì private key của CA là tài sản có rủi ro cao. Trong hệ thống nghiêm túc, CA key thường cần cơ chế bảo vệ và chính sách vận hành riêng.

Vị trí công nghệ của EJBCA là CA platform hoàn chỉnh. Nó phù hợp khi cần mô phỏng hoặc vận hành CA có policy, profile, enrollment, thu hồi, OCSP, API và tích hợp bảo mật. Đổi lại, độ phức tạp cao hơn OpenSSL và XCA. Người vận hành phải hiểu cấu hình CA, profile, quyền quản trị, lưu khóa và quy trình thu hồi.

6.5 Let's Encrypt và Certbot

Let's Encrypt là CA công cộng miễn phí, tự động và mở do Internet Security Research Group vận hành [24]. Nó thuộc Web PKI công cộng, không phải CA nội bộ cho tên riêng hoặc định danh doanh nghiệp. Certbot là ACME client phổ biến để lấy, cài đặt, gia hạn, thu hồi, xóa và liệt kê chứng chỉ [25].

ACME định nghĩa cách client tự động làm việc với CA. RFC 8555 mô tả luồng điển hình: tạo hoặc dùng account, tạo order, lấy authorization, hoàn thành challenge, finalize order bằng CSR và tải chứng chỉ đã cấp [17]. Vì vậy, ACME chuyển nhiều thao tác cấp TLS certificate thành giao thức tự động.

Các challenge ACME chứng minh quyền kiểm soát identifier. HTTP-01 dùng file dưới đường dẫn `/.well-known/acme-challenge/` và chỉ chạy trên cổng 80. DNS-01 dùng TXT record dưới `_acme-challenge`. TLS-ALPN-01 chạy qua TLS trên cổng 443 và dùng ALPN riêng [41]. DNS-01 hỗ trợ wildcard certificate, còn HTTP-01 và TLS-ALPN-01 không hỗ trợ wildcard theo tài liệu Let's Encrypt [41].

Certbot cung cấp các plugin xác thực và cài đặt. Apache và Nginx plugin có thể lấy và cài chứng chỉ. Webroot plugin đặt challenge file trong web root hiện có. Standalone plugin chạy web server tạm thời và cần truy cập cổng 80. DNS plugin dùng TXT record và cần thiết cho wildcard certificate [25].

Gia hạn là điểm mạnh vận hành của Certbot. Lệnh `certbot renew` kiểm tra các chứng chỉ đã biết và gia hạn chứng chỉ gần hết hạn. Hầu hết cài đặt Certbot có scheduled task chạy `certbot renew`. Certbot cũng hỗ trợ hook trước, sau và sau khi triển khai để dừng, khởi động hoặc nạp lại dịch vụ [25].

Giới hạn chính của mô hình này là nó phụ thuộc vào quyền kiểm soát tên miền. Let's Encrypt cũng áp dụng rate limit. Tài liệu hiện hành nêu giới hạn 50 chứng chỉ mới cho mỗi registered domain trong 7 ngày và 5 chứng chỉ cho cùng exact identifier set trong 7 ngày [42]. Do đó, tự động hóa cần môi trường staging và cấu hình gia hạn cẩn thận.

6.6 Vault PKI

Vault PKI Secrets Engine thuộc nhóm internal PKI và secret management. Tài liệu HashiCorp mô tả engine này sinh chứng chỉ X.509 động. Dịch vụ có thể lấy chứng chỉ qua cơ chế xác thực và phân quyền của Vault, thay vì luồng CSR thủ công với CA ngoài [28].

Vault PKI thường dùng role để ràng buộc chính sách phát hành. Role có thể giới hạn domain được phép, cho phép subdomain và đặt TTL tối đa. Chứng chỉ được phát hành bằng cách gọi endpoint `issue` theo tên role, và Vault trả về certificate, issuing CA, private key, key type và serial number [43].

Điểm mạnh của Vault PKI là API-driven issuance và TTL ngắn. Tài liệu HashiCorp nhấn mạnh certificate TTL ngắn giúp giảm phụ thuộc vào thu hồi và làm CRL nhỏ hơn

trong môi trường nhiều workload [28]. Mỗi instance dịch vụ có thể nhận chứng chỉ riêng, giúp giảm rủi ro khi phải xoay vòng một chứng chỉ dùng chung.

Vault PKI phù hợp với internal TLS, mTLS giữa service và hạ tầng ứng dụng. Nó không thay thế public CA vì chứng chỉ do Vault phát hành chỉ được tin bởi các hệ thống đã cài trust anchor tương ứng. Nó cũng không phải CA platform enterprise đầy đủ theo nghĩa EJBCA hoặc Dogtag. Vai trò chính của Vault là phát hành bí mật và chứng chỉ ngắn hạn theo policy.

6.7 OAuth/OIDC và PKI

OAuth 2.0 là authorization framework cho dịch vụ HTTP. RFC 6749 định nghĩa các vai trò như resource owner, resource server, client và authorization server. Client nhận access token có scope và thời hạn, thay vì dùng trực tiếp mật khẩu của resource owner [29].

OpenID Connect bổ sung identity layer trên OAuth 2.0. OpenID Connect Core định nghĩa ID Token là JWT chứa claim về sự kiện xác thực người dùng. ID Token phải được ký bằng JWS và có thể được mã hóa thêm bằng JWE [30].

JWT là định dạng compact để mang claim giữa hai bên. RFC 7519 mô tả JWT Claims Set và các claim như `iss`, `sub`, `aud`, `exp` và `iat`. Nội dung JWT có thể được bảo vệ bằng JWS hoặc JWE [33]. JWS cung cấp chữ ký số hoặc MAC trên payload [31].

JWKS là tập các JSON Web Key. RFC 7517 định nghĩa JWK Set là JSON object có mảng `keys`. Trường `kid` giúp bên xác minh chọn đúng khóa khi có nhiều khóa hoặc khi xoay khóa [32]. Vì vậy, resource server có thể lấy public key từ JWKS để kiểm tra chữ ký token.

Ranh giới với PKI cần được giữ rõ. PKI quản lý ràng buộc public key với identity qua certificate và chuỗi CA. OAuth/OIDC quản lý authorization, identity token và xác minh token. Điểm giao là TLS certificate bảo vệ kênh đến IdP hoặc API, IdP ký token bằng private key, và service xác minh token bằng public key.

Một điểm giao trực tiếp hơn là mutual TLS trong OAuth. RFC 8705 định nghĩa `tls_client_auth` cho client authentication dựa trên X.509 mutual TLS và certificate-bound access token. Với token dạng này, resource server phải so khớp certificate trình bày qua mTLS với binding trong token [44].

6.8 Dogtag và FreeIPA

Dogtag Certificate System là enterprise-class open source Certificate Authority. Tài liệu Dogtag mô tả nó là full PKI platform quản lý vòng đời chứng chỉ, gồm issuance, revocation, retrieval và profiles. Nó cũng hỗ trợ CRL generation, OCSP responder, SCEP và local Registration Authority [27].

Dogtag phù hợp với môi trường cần CA nội bộ có chức năng enterprise. So với EJBCA, Dogtag thường được nhìn trong hệ sinh thái Linux doanh nghiệp và Red Hat. Nó mạnh hơn OpenSSL/XCA vì có vòng đời chứng chỉ, profile, thu hồi và dịch vụ PKI. Đổi lại, vận hành Dogtag đòi hỏi hiểu nhiều thành phần hệ thống hơn.

FreeIPA là hệ thống identity management cho môi trường Linux/UNIX. Tài liệu FreeIPA mô tả nó tích hợp identity, authentication, authorization và account management. Nó quản lý user, group, host và các đối tượng bảo mật mạng. FreeIPA dùng Dogtag Certificate System để cung cấp CA và RA functions [45].

Vị trí của FreeIPA khác Dogtag thuần PKI. FreeIPA đặt certificate service trong hệ định danh gồm Kerberos, LDAP, DNS, web UI và CLI [45]. Nó phù hợp khi tổ chức muốn quản lý user, host, service và certificate trong cùng một miền nội bộ. Mức phức tạp cao hơn công cụ local, nhưng đổi lại có tích hợp định danh tập trung.

6.9 So sánh hệ sinh thái

Table 6.1: So sánh một số công cụ và hệ thống PKI mã nguồn mở

Công cụ	Nhóm	Vai trò chính	Môi trường	CA	API	Thu hồi	Định danh	Độ phức tạp	Giới hạn chính
OpenSSL	Toolkit	Thao tác khóa, CSR, chứng chỉ	Local, script	Rất cơ bản	CLI	Qua CRL khi cấu hình	Không	Thấp	Không có RA workflow hoặc audit tập trung.
XCA	Local management	Quản lý CA và certificate store cục bộ	Lab, học tập	CA cục bộ	Thấp	Có CRL	Không	Thấp đến vừa	Không phải enterprise CA hoặc API-first system.
EJBCA	CA platform	Quản trị CA có profile và lifecycle	CA nội bộ, CA có policy	Đầy đủ	REST, ACME	OCSP, CRL	RA, OAuth cho REST	Cao	Cần cấu hình profile, quyền và vận hành khóa.
Let's Encrypt/Certbot	Public	Tự động hóa TLS public	Web PKI công cộng	CA công cộng	ACME, plugin	ACME revocation	Domain validation	Vừa	Phụ thuộc domain control và rate limit.
Vault PKI	Internal PKI	Cấp chứng chỉ nội bộ ngắn hạn	Service nội bộ, mTLS	CA nội bộ	HTTP API, role	CRL, TTL ngắn	Vault auth/policy	Vừa	Không thay public CA hoặc CA enterprise.
OAuth/OIDC stack	Identity	Xác minh token và identity	IdP, API	Không phải CA	JWKS, token validation	Không quản lý chứng chỉ	Rất mạnh	Vừa đến cao	Quản lý token, không quản lý PKI.

Công cụ	Nhóm	Vai trò chính	Môi trường	CA	API	Thu hồi	Định danh	Độ phức tạp	Giới hạn chính
Dogtag/FreeIPA	Enterprise identity + PKI	CA và certificate trong identity domain	Linux enterprise	Đầy đủ trong Dogtag	Web, CLI	CRL, OCSP	LDAP, Kerberos	Cao	Phức tạp và gắn với hệ sinh thái Linux.

Có thể nhìn hệ sinh thái này theo tầng. OpenSSL và XCA nằm ở tầng local PKI và kiểm thử. Certbot và Let's Encrypt giải quyết tự động hóa TLS trong Web PKI công cộng. EJBCA đại diện cho CA platform hoàn chỉnh, nơi policy và vòng đời chứng chỉ là trung tâm.

Vault PKI phù hợp với chứng chỉ nội bộ ngắn hạn cho application infrastructure. OAuth/OIDC không phải PKI platform, nhưng dùng chữ ký số, JWKS, mTLS và TLS certificate trong xác thực hiện đại. Dogtag và FreeIPA đặt PKI vào hệ identity enterprise, nơi user, host, service và certificate được quản lý cùng nhau.

Chapter 7

Thách thức và xu hướng

PKI vẫn là nền tảng để ràng buộc public key với danh tính. Khi số lượng certificate tăng, rủi ro chính chuyển sang kiểm soát vòng đời, niềm tin và khóa riêng.

7.1 Thách thức hiện tại

Thách thức đầu tiên là quản lý vòng đời chứng chỉ. Một certificate có thể được cấp đúng nhưng gây lỗi khi hết hạn, không được gia hạn, hoặc bị bỏ quên trong một trust store cũ. Ở quy mô lớn, tổ chức phải kiểm kê certificate, chủ sở hữu, CA phát hành, thời hạn và mục đích sử dụng. Thiếu kiểm kê làm xoay vòng khóa và thay thế certificate trở thành thao tác thủ công dễ sai.

Thu hồi chứng chỉ cũng có giới hạn thực tế. CRL được định nghĩa trong profile X.509, nhưng có thể lớn và có độ trễ phân phối [5]. OCSP cho phép kiểm tra trạng thái trực tuyến, nhưng lại phụ thuộc vào responder và chính sách xử lý khi dịch vụ trạng thái không sẵn sàng [6]. Vì vậy, thu hồi không phải cơ chế tức thời tuyệt đối.

Niềm tin vào CA là rủi ro tập trung. Nếu CA cấp sai hoặc bị xâm phạm, trình duyệt và hệ điều hành phải dựa vào root store, chính sách CA và cơ chế distrust để giới hạn thiệt hại. CA/B Forum Baseline Requirements đặt yêu cầu cho CA publicly-trusted [4].

7.2 Tự động hóa và giám sát

Xu hướng chính là giảm thao tác thủ công. ACME chuẩn hóa cách client tạo order, chứng minh quyền kiểm soát identifier, gửi CSR và nhận certificate [17]. Certificate lifecycle management mở rộng ý tưởng này sang kiểm kê, cấp phát, gia hạn, thu hồi và audit. Short-lived certificate cũng giảm khoảng thời gian rủi ro khi khóa hoặc certificate bị lộ.

Tự động hóa cần đi kèm policy. Nếu hệ thống cấp certificate không kiểm soát được ai có quyền yêu cầu, tên nào được cấp, hoặc profile nào được dùng, lỗi thủ công chỉ chuyển thành lỗi tự động. Vì vậy, inventory, phân quyền và audit là điều kiện vận hành, không

Table 7.1: Một số hướng xử lý thách thức PKI

Hướng	Giảm rủi ro	Giới hạn
ACME và CLM	Giảm lỗi cấp phát và gia hạn thủ công	Cần policy, phân quyền và inventory.
Short-lived certificate	Giảm thời gian hiệu lực khi có sự cố	Phụ thuộc tự động hóa gia hạn ổn định.
Certificate Transparency	Tăng khả năng phát hiện certificate cấp sai	Không thay thế validation hoặc revocation.
Crypto-agility	Cho phép thay thuật toán và profile	Cần hỗ trợ từ CA, HSM, TLS stack và client.

phải phần phụ.

Bảng 7.1 tóm tắt vai trò của một số hướng xử lý hiện nay.

Certificate Transparency bổ sung lớp giám sát cho Web PKI. RFC 9162 mô tả CT bằng các log append-only dựa trên Merkle Tree, giúp certificate đã cấp có thể được audit công khai [15]. CT làm tăng minh bạch và hỗ trợ phát hiện mis-issuance. Nó không tự xác thực domain, không thu hồi certificate và không thay thế chính sách CA.

7.3 PKI trong hệ thống hiện đại

PKI ngày càng gắn với machine identity. Service, container, workload và thiết bị IoT đều cần danh tính mật mã để kết nối an toàn. Khác với người dùng, những thực thể này có số lượng lớn, vòng đời ngắn và thường được tạo tự động. Điều đó làm certificate inventory và policy trở thành phần của hạ tầng vận hành.

Trong kiến trúc zero trust và microservice, mTLS dùng certificate để xác thực hai chiều giữa các service. Cách làm này giảm phụ thuộc vào vị trí mạng, nhưng tăng yêu cầu đối với CA nội bộ, phân phối trust anchor và xoay vòng private key. Internal PKI vì vậy cần tích hợp với quản trị danh tính, secret management và audit thay vì chỉ phát hành certificate rời rạc.

7.4 Post-quantum

Máy lượng tử đủ mạnh sẽ đe dọa RSA, Diffie-Hellman, ECDSA và ECDH vì các hệ này dựa trên bài toán phân tích số hoặc logarit rời rạc. NIST đã công bố các chuẩn hậu lượng tử chính năm 2024, gồm KEM và thuật toán chữ ký số [46]. Tác động tới PKI không chỉ là thay thuật toán. Certificate và chữ ký có thể lớn hơn, chain validation phức tạp hơn, còn CA, HSM, TLS stack và trust store phải cùng hỗ trợ profile mới.

Vì chưa nên dự đoán chắc mốc thời gian cho khả năng phá vỡ lượng tử, hướng thực tế là crypto-agility. Một hệ PKI cần thay được thuật toán, profile và policy mà không làm

gián đoạn cấp phát, xác minh và thu hồi certificate. Đây là lý do PKI phải tiến hóa ở tầng vận hành, không chỉ ở lựa chọn thuật toán.

Chapter 8

Kết luận

8.1 Tổng kết

Báo cáo đã đi từ nền tảng mật mã khóa công khai ở Chương 2 đến bài toán phân phối khóa và ràng buộc danh tính trong hạ tầng PKI ở Chương 3. Trên cơ sở đó, Chương 4 trình bày chứng chỉ số, X.509 và các chuẩn liên quan, trước khi Chương 5 phân tích cách PKI xuất hiện trong HTTPS, chữ ký số, S/MIME, VPN và code signing.

Các chương cuối đặt PKI vào bối cảnh vận hành. Chương 6 khảo sát các công cụ và nền tảng mã nguồn mở, từ OpenSSL, XCA đến EJBCA, Certbot, Vault PKI và FreeIPA. Chương 7 tổng hợp các thách thức về vòng đời chứng chỉ, thu hồi, niềm tin vào CA, tự động hóa và crypto-agility. Kết luận chính là PKI không chỉ gồm thuật toán hay certificate. Nó là hệ thống kết hợp mật mã, chính sách tin cậy, quy trình vận hành và công cụ quản trị.

8.2 Ý nghĩa và lời cảm ơn

Qua quá trình thực hiện báo cáo, nhóm hiểu rõ hơn cách public key được gắn với danh tính thông qua CA và certificate chain. Báo cáo cũng giúp nhóm nhìn thấy vai trò của revocation, trust store, certificate lifecycle và tự động hóa trong một hệ PKI thực tế. Những nội dung này tạo nền tảng để tiếp tục đọc tài liệu kỹ thuật hoặc xây dựng mô hình thử nghiệm PKI ở mức có kiểm soát.

Nhóm xin chân thành cảm ơn PGS. TS. Nguyễn Linh Giang đã hướng dẫn, cung cấp định hướng môn học và hỗ trợ nhóm trong quá trình thực hiện báo cáo.

Tài liệu tham khảo

- [1] J. G. Steiner, B. C. Neuman, and J. I. Schiller, “Kerberos: An authentication service for open network systems,” in *Proceedings of the USENIX Winter Conference*, 1988, pp. 191–202.
- [2] R. L. Rivest, A. Shamir, and L. Adleman, “A method for obtaining digital signatures and public-key cryptosystems,” *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978. DOI: [10.1145/359340.359342](https://doi.org/10.1145/359340.359342)
- [3] P. R. Zimmermann, *PGP: Source code and internals*, 1995.
- [4] CA/Browser Forum, *Baseline requirements for the issuance and management of publicly-trusted certificates*, 2023. [Online]. Available: <https://cabforum.org/baseline-requirements-documents/>
- [5] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, “Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile,” IETF, RFC 5280, 2008. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5280>
- [6] S. Santesson, M. Myers, R. Ankney, A. Malpani, S. Galperin, and C. Adams, “X.509 Internet Public Key Infrastructure Online Certificate Status Protocol – OCSP,” IETF, RFC 6960, 2013. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6960>
- [7] D. E. 3rd, “Transport Layer Security (TLS) Extensions: Extension Definitions,” IETF, RFC 6066, 2011, Section 8: Certificate Status Request (OCSP Stapling). [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6066>
- [8] P. Saint-Andre and J. Hodges, “Representation and Verification of Domain-Based Application Service Identity within Internet Public Key Infrastructure Using X.509 Certificates in the Context of Transport Layer Security (TLS),” IETF, RFC 6125, 2011. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6125>
- [9] M. Nystrom and B. Kaliski, “PKCS #10: Certification Request Syntax Specification Version 1.7,” IETF, RFC 2986, 2000. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc2986>

- [10] CA/Browser Forum, *Guidelines for the issuance and management of extended validation certificates*, 2023. [Online]. Available: <https://cabforum.org/extended-validation/>
- [11] J. Schaad, B. Ramsdell, and S. Turner, “Secure/Multipurpose Internet Mail Extensions (S/MIME) Version 4.0 Message Specification,” IETF, RFC 8551, 2019. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8551>
- [12] R. Housley, “Cryptographic Message Syntax (CMS),” IETF, RFC 5652, 2009. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5652>
- [13] K. Moriarty, M. Nystrom, S. Parkinson, A. Rusch, and M. Scott, “PKCS #12: Personal Information Exchange Syntax v1.1,” IETF, RFC 7292, 2014. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7292>
- [14] B. Laurie, A. Langley, and E. Kasper, “Certificate Transparency,” IETF, RFC 6962, 2013. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6962>
- [15] B. Laurie, A. Langley, E. Kasper, E. Messeri, and R. Stradling, “Certificate Transparency Version 2.0,” IETF, RFC 9162, 2021. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9162>
- [16] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” IETF, RFC 8446, 2018. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8446>
- [17] R. Barnes, J. Hoffman-Andrews, D. McCarney, and J. Kasten, “Automatic Certificate Management Environment (ACME),” IETF, RFC 8555, 2019. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8555>
- [18] C. Adams, P. Cain, D. Pinkas, and R. Zuccherato, “Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP),” IETF, RFC 3161, 2001. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc3161>
- [19] C. Kaufman, P. Hoffman, Y. Nir, P. Eronen, and T. Kivinen, “Internet Key Exchange Protocol Version 2 (IKEv2),” IETF, RFC 7296, 2014. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7296>
- [20] Microsoft, *Cryptography tools – win32 apps*, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/seccrypto/cryptography-tools>
- [21] Apple Developer Documentation, *Code signing services*, 2026. [Online]. Available: <https://developer.apple.com/documentation/security/code-signing-services>
- [22] Trung tâm Chứng thực điện tử quốc gia, *Trang thông tin điện tử trung tâm chứng thực điện tử quốc gia*, 2026. [Online]. Available: <https://neac.gov.vn>
- [23] C. Hohnstaedt, *Xca: Certificate and key management*, 2026. [Online]. Available: <https://hohnstaedt.de/xca/>

- [24] Let's Encrypt, *Documentation*, 2026. [Online]. Available: <https://letsencrypt.org/docs/>
- [25] Electronic Frontier Foundation, *Certbot user guide*, 2026. [Online]. Available: <https://eff-certbot.readthedocs.io/en/stable/using.html>
- [26] Keyfactor, *Ejbca documentation*, 2026. [Online]. Available: <https://docs.keyfactor.com/ejbca/>
- [27] Dogtag PKI Team, *Dogtag pki documentation*, 2026. [Online]. Available: https://www.dogtagpki.org/wiki/PKI_Main_Page
- [28] HashiCorp, *Pki secrets engine*, 2026. [Online]. Available: <https://developer.hashicorp.com/vault/docs/secrets/pki>
- [29] D. Hardt, “The OAuth 2.0 Authorization Framework,” IETF, RFC 6749, 2012. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6749>
- [30] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, *OpenID Connect Core 1.0*, 2014. [Online]. Available: https://openid.net/specs/openid-connect-core-1_0.html
- [31] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Signature (JWS),” IETF, RFC 7515, 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7515>
- [32] M. Jones, “JSON Web Key (JWK),” IETF, RFC 7517, 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7517>
- [33] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF, RFC 7519, 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [34] OpenSSL Project, *openssl-x509: Certificate display and signing command*, 2026. [Online]. Available: <https://docs.openssl.org/master/man1/openssl-x509/>
- [35] OpenSSL Project, *openssl-verify: Certificate verification command*, 2026. [Online]. Available: <https://docs.openssl.org/master/man1/openssl-verify/>
- [36] Keyfactor, *Ejbca certificate profiles overview*, 2026. [Online]. Available: <https://docs.keyfactor.com/ejbca/latest/certificate-profiles-overview>
- [37] Keyfactor, *Ejbca end entity profiles overview*, 2026. [Online]. Available: <https://docs.keyfactor.com/ejbca/latest/end-entity-profiles-overview>
- [38] Keyfactor, *Ejbca rest interface*, 2026. [Online]. Available: <https://docs.keyfactor.com/ejbca/latest/ejbca-rest-interface>
- [39] Keyfactor, *Ejbca acme*, 2026. [Online]. Available: <https://docs.keyfactor.com/ejbca/latest/acme>
- [40] Keyfactor, *Ejbca ocsf*, 2026. [Online]. Available: <https://docs.keyfactor.com/ejbca/latest/ocsf>

- [41] Let’s Encrypt, *Challenge types*, 2026. [Online]. Available: <https://letsencrypt.org/docs/challenge-types/>
- [42] Let’s Encrypt, *Rate limits*, 2026. [Online]. Available: <https://letsencrypt.org/docs/rate-limits/>
- [43] HashiCorp, *Build your own certificate authority with vault*, 2026. [Online]. Available: <https://developer.hashicorp.com/vault/docs/secrets/pki/quick-start-root-ca>
- [44] B. Campbell, J. Bradley, N. Sakimura, and T. Lodderstedt, “OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens,” IETF, RFC 8705, 2020. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8705>
- [45] FreeIPA Project, *About freeipa*, 2026. [Online]. Available: <https://www.freeipa.org/page/About>
- [46] National Institute of Standards and Technology, “Post-Quantum Cryptography: FIPS 203, FIPS 204, FIPS 205,” NIST, Tech. Rep., 2024. [Online]. Available: <https://csrc.nist.gov/projects/post-quantum-cryptography>